

claude-windows / f81fb2b5-e34b-427e-a8a1-a37eb4539a49

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\f81fb2b5-e34b-427e-a8a1-a37eb4539a49\subagents\workflows\wf_56274ff0-603\agent-ac295c673202dba39.jsonl`
- SHA-256: `207df0d91b813cfa453d9e8600f98c0368c7e4c13168335bf84fbe0dbdac08a4`
- Source modified: `2026-06-21T02:41:19+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_56274ff0-603`
- session_id: `f81fb2b5-e34b-427e-a8a1-a37eb4539a49`
```

Transcript

1. user (2026-06-21T02:36:38.876Z)

You are designing a tracked project: **compute-decoupled rally serving** ? one CORE engine (detect + stitch) that runs IDENTICALLY across deployment profiles, with the input source + compute target chosen by a startup/setup config. Be thorough, code-grounded, and honest (mark exists/partial/new).

OWNER REQUIREMENTS (verbatim, 2026-06-21):

- (a) Host a SERVICE that operates on small LOCAL uploaded files (<2GB) AND can load from **gdrive** or a bucket, and **spins up Cloud Run** to get the rally segments AND the stitching ? because stitching is also compute-intensive, so run it on Cloud Run so its cost is accounted for.
- (b) Support a TRULY LOCAL mode: running on a local machine with videos on local drives/USB and a local GPU, run the inference AND stitching COMPLETELY locally.
- (c) CRITICAL: the CORE of the tool (detecting + stitching) must be UNAFFECTED across modes; the input sources (the decoupled effort) are orchestrated via a "setup/startup" config or other means.
- Deliverables the human will author from your output: a thorough design, an ADR, a project directory with **runlogs + sample runs for posterity**, an execution plan, and a **very rigorous testing strategy**.

Context: this is the productionization of DECOUPLED_COMPUTE (now archived, delivered on GCP). ADR-012 = GPU-native; Cloud Run+GPU scale-to-zero is the serving model; ADR-013 = GCP is the sole compute stack. The decoupled worker `python -m backend.worker {infer|train}` already runs over gcs:// buckets; the native WASB runner (detector_impl="native") + storage backends + async jobs control plane + the per-video workspace helper already exist. The PLATFORM_ARCHITECTURE strategy doc already names a ComputeBackend registry (local | hosted | byo_worker) ? realize that.

Here is the complete code-grounded map of the core engine, seams, control plane, and testing surface (JSON):

```
[
  {
    "area": "Core Rally Engine (Detection + Stitching) ? Local vs Cloud Deployment",
    "findings": [
      {
        "topic": "Core Contract: Detection Phase (GPU Compute)",
        "detail": "DetectorRunner.run_predict(video_path: str, output_dir: str, video_id: str)
-> Optional[str] ? returns absolute Windows/native path to Frame,Visibility,X,Y CSV. Three
concrete implementations: WasbRunner (WSL+conda on Windows), NativeWasbRunner (Linux GPU box),
StubDetectorRunner (CPU mock). All emit identical CSV schema. The contract mandates trajectory
shape uniformity so trajectory_to_action_windows() is model-agnostic.",
        "code_ref": "backend/pipeline/detectors/base.py:164-192",
        "kind": "core-contract",
        "reuse_status": "exists"
      },
    ],
  },
]
```

```

{
    "topic": "Core Contract: Windowing Phase (CPU Pure Function)",
    "detail": "TrackNetRunner.trajectory_to_action_windows(points: List[TrajectoryPoint],
fps, frame_width, velocity_thresh, merge_gap, ...) -> List[Dict] ? deterministic pure function:
trajectory points + thresholds -> candidate rally windows (start, end, peak_velocity,
mean_velocity, active_frame_count, track_id_count). Shared @staticmethod reused by ALL runners
(WasbRunner, NativeWasbRunner, StubDetectorRunner, TrackNetRunner). Zero GPU dependency.
Behaviour-identical local vs cloud.",
    "code_ref": "backend/pipeline/detectors/tracknet_runner.py:281-298 (signature), entire
trajectory_to_action_windows ~600 lines",
    "kind": "core-contract",
    "reuse_status": "exists"
},
{
    "topic": "Core Contract: Stitching Phase (FFmpeg + Watermark)",
    "detail": "VideoCompiler.compile_highlights(raw_video_path: str, segments:
List[Tuple[float, float]], output_filename: str) -> str ? pure filesystem + subprocess
composition: (1) _merge_overlapping() dedupes segments, (2) _get_video_duration() via ffprobe,
(3) _watermark_filter() builds optional drawtext/overlay, (4) _trim_one_segment() per-clip re-
encode with libx264+watermark, (5) concat demuxer stitches. Output deterministic given same
input. Behaviour-identical local vs cloud.",
    "code_ref": "backend/pipeline/stitcher/compiler.py:303-414 (compile_highlights main),
190-301 (_trim_one_segment)",
    "kind": "core-contract",
    "reuse_status": "exists"
},
{
    "topic": "Detection Seam: GPU Device Abstraction",
    "detail": "TrajectoryHybridSegmenter._resolve_runner(idx_cfg) -> (DetectorRunner,
extras_dict) ? honours RALLY_DETECTOR_IMPL env var or indexing.detector_impl config: 'stub' ->
StubDetectorRunner (CPU, no GPU/WSL), 'native' -> NativeWasbRunner (Linux GPU box), 'auto'
(default) -> WasbRunner (WSL on Windows). The seam is the ONLY place deployment mode is chosen;
callers (process_video) are agnostic. Detector-specific extras (detector_impl, weights_path)
threaded into detection_params for DB storage.",
    "code_ref": "backend/pipeline/segmenters/trajectory_hybrid.py:65-86 (_resolve_runner),
193-199 (healthcheck gate)",
    "kind": "coupling-point",
    "reuse_status": "exists"
},
{
    "topic": "Detection Seam: Healthcheck Gate (Environment Readiness)",
    "detail": "Every runner implements healthcheck() -> (bool, str) ? cheap, non-raising
check of env readiness (WSL + conda env + GPU visible for WasbRunner; torch importable for
NativeWasbRunner; trivial True for StubRunner). Called ONCE per video before expensive
inference. Failure returns empty result [] (no rallies), with message logged. Ensures fail-fast
on misconfiguration.",
    "code_ref": "backend/pipeline/segmenters/trajectory_hybrid.py:227-231 (healthcheck
call),
backend/pipeline/detectors/{wasb_runner,native_wasb_runner,stub_runner}.py:healthcheck()",
    "kind": "coupling-point",
    "reuse_status": "exists"
},
{
    "topic": "FILESYSTEM COUPLING: Relative Output Paths (Hardcoded 'output/')",
    "detail": "trajectory_hybrid.py:237 ? `out_dir = os.path.join('output', self.family)`
(relative to CWD). WasbRunner.run_predict() resolves via os.path.abspath(output_win_dir) before
WSL /mnt translation, but the segmenter PASSES the relative path. On a cloud box with a
different working directory or no 'output/' sibling, this path BREAKS. Same issue: handoff_dir =
os.path.join('output', f'chunks_{provider_name}_handoff') line 313. On cloud, must resolve to an
absolute path or configurable base.",
    "code_ref": "backend/pipeline/segmenters/trajectory_hybrid.py:237, 313",
    "kind": "config-seam",
    "reuse_status": "exists"
},
{

```

```

        "topic": "FILESYSTEM COUPLING: CSV Staging and Disk Paths (WSL /mnt Translation)",
        "detail": "WasbRunner stages video into WSL filesystem (~clips/) and stages
wasb_infer.py into WASB repo src/ for import access. On Windows, the WSL path via
to_wsl_mnt_path(video_path) -> /mnt/c/Users/... works. On Linux (NativeWasbRunner), there is NO
/mnt translation ? native shutil.copy is used. The CSV is written to output_dir (absolute path).
On cloud, if output_dir is a relative path passed by the segmenter, the CSV lands in a place the
segmenter can't find (different CWD).",
        "code_ref": "backend/pipeline/detectors/wasb_runner.py:88-104 (_stage_video), 106-111
(_stage_infer_script), 125-134 (run_predict path resolution), native_wasb_runner.py:150-165
(native run_predict, no WSL, uses os.path.abspath)",
        "kind": "coupling-point",
        "reuse_status": "exists"
    },
    {
        "topic": "DEVICE ASSUMPTION: GPU/CUDA in WasbRunner and NativeWasbRunner",
        "detail": "WasbRunner healthcheck() runs `torch.cuda.is_available()` inside WSL conda
env ? assumes Windows + WSL2 + NVIDIA GPU. NativeWasbRunner healthcheck() runs native python +
torch, device defaults to 'cuda' (overridable by WASB_DEVICE env var or config). On a CPU-only
box, both fail healthcheck and return empty result. On cloud without GPU,
NativeWasbRunner.device='cpu' must be set explicitly (environment or config).",
        "code_ref": "backend/pipeline/detectors/wasb_runner.py:72-86 (healthcheck),
native_wasb_runner.py:125-148 (healthcheck, device check)",
        "kind": "coupling-point",
        "reuse_status": "exists"
    },
    {
        "topic": "DEVICE ASSUMPTION: WSL2 Required for Windows WasbRunner",
        "detail": "WasbRunner invokes `wsl -d Ubuntu bash -c` ? requires Windows + WSL2 with a
Linux distro and conda environment pre-configured. If WSL is absent, healthcheck returns False
with message '`wsl` not found ? Windows + WSL2 required.' On cloud (Linux), this runner is never
instantiated ? NativeWasbRunner is chosen instead.",
        "code_ref": "backend/pipeline/detectors/wasb_runner.py:72-86 (healthcheck
FileNotFoundError), base.py:106-111 (_wsl_bash)",
        "kind": "coupling-point",
        "reuse_status": "exists"
    },
    {
        "topic": "FILESYSTEM COUPLING: Hardcoded Relative Paths in Stitcher Input",
        "detail": "VideoCompiler.__init__ takes output_dir (absolute path expected). But callers
(e.g., yolo_hybrid.py:407 `chunks_dir = os.path.join('output', 'chunks_yolo')`) pass relative
paths. compile_highlights() calls _get_video_duration(video_path) which shells ffprobe ? ffprobe
succeeds if video_path is absolute or relative to CWD. If CWD differs across deployments,
relative paths break.",
        "code_ref": "backend/pipeline/stitcher/compiler.py:17-25 (__init__), 107-123
(_get_video_duration), backend/pipeline/segmenters/yolo_hybrid.py:407",
        "kind": "config-seam",
        "reuse_status": "exists"
    },
    {
        "topic": "FILESYSTEM COUPLING: Watermark Font Path (Absolute vs System-Dependent)",
        "detail": "compiler.py:14 ? `_BUNDLED_FONT = Path(__file__).resolve().parents[2] /
'assets' / 'fonts' / 'RobotoSlab-Regular.ttf'`. Fallback to system fontconfig if bundled font
missing. On Windows, font resolution is 'Sans' (system-dependent). On cloud Linux, may be
different or absent. _ff_quote() handles Windows drive-letter colons (C:/ -> C\\:/), but system
font availability is environment-dependent. Config-overridable via
stitching.watermark.font_path.",
        "code_ref": "backend/pipeline/stitcher/compiler.py:13-14 (_BUNDLED_FONT), 159-188
(_watermark_filter, font precedence)",
        "kind": "coupling-point",
        "reuse_status": "exists"
    },
    {
        "topic": "FILESYSTEM COUPLING: FFmpeg Binary Availability",
        "detail": "All GPU compute and stitching uses subprocess.run(['ffmpeg', ...],
['ffprobe', ...]) ? assumes ffmpeg/ffprobe are in PATH. On cloud, these must be installed + on

```

```

PATH. No fallback; failure is fatal. No version pinning in code ? assumes compatible version.",
    "code_ref": "backend/pipeline/stitcher/compiler.py:111-119 (_get_video_duration),
248-271 (_trim_one_segment ffmpeg cmd), 386-393 (concat cmd), backend/utls/video.py:10-18
(ffprobe), 34-45 (ffmpeg)",
    "kind": "coupling-point",
    "reuse_status": "exists"
},
{
    "topic": "FILESYSTEM COUPLING: Temporary Directory and Scratch Space",
    "detail": "WasbRunner uses WSL stage dir (`wsl_stage_dir` config, default `~/clips`) ?
subject to WSL filesystem space. NativeWasbRunner uses config scratch_dir or output_dir fallback
for frames. VideoCompiler.compile_highlights() creates
tempfile.TemporaryDirectory(dir=output_dir) ? assumes output_dir is writable and has sufficient
space. No cloud-aware staging to NVMe or ephemeral storage.",
    "code_ref": "backend/pipeline/detectors/wasb_runner.py:39 (wsl_stage_dir config),
142-147 (disk guard), native_wasb_runner.py:66, 87 (scratch_dir resolution), compiler.py:340
(tempfile in output_dir)",
    "kind": "coupling-point",
    "reuse_status": "exists"
},
{
    "topic": "AI Handoff Seam: Provider + API Key Wiring",
    "detail": "trajectory_hybrid.py:159-181 ? skip_ai_handoff config (default False) gates
AI path. If enabled, requires api_key from environment (GEMINI_API_KEY, etc.) or error + empty
result. Provider chosen via ai_provider config (default 'gemini'), resolved from
provider_registry. Handoff clips extracted to 'output/chunks_{provider_name}_handoff' (relative
path). Each window becomes padded clip -> AI analyze -> segment extraction. Deterministic given
same provider API + model ? but provider latency + queueing are not.",
    "code_ref": "backend/pipeline/segmenters/trajectory_hybrid.py:158-181 (provider setup),
290-352 (AI handoff Phase 3)",
    "kind": "coupling-point",
    "reuse_status": "exists"
},
{
    "topic": "Detection + Windowing Behaviour Identity (Regression Test Seam)",
    "detail": "StubDetectorRunner with replay_csv=path to golden CSV replicates exact
trajectory (byte-for-byte same CSV -> byte-for-byte same candidate windows). Enables end-to-end
regression testing on CPU without GPU/WSL. Used in tests via detector_impl='stub' and
RALLY_DETECTOR_IMPL=stub env var. The windowing algorithm is deterministic (velocity, merging,
thresholds all floating-point with no randomness) ? same trajectory + same config = identical
windows.",
    "code_ref": "backend/pipeline/detectors/stub_runner.py:64-122 (StubDetectorRunner),
105-108 (replay_csv path), trajectory_hybrid.py:75-81 (_resolve_runner stub case)",
    "kind": "config-seam",
    "reuse_status": "exists"
},
{
    "topic": "Candidate Persistence and AI Handoff Gating",
    "detail": "trajectory_hybrid.py:276-285 ? raw candidate windows persisted to DB
(candidate_segments table) even if not selected for handoff. _select_for_handoff() (default: all
windows) and _enrich_candidate_stats() (base: 4 motion keys) are fusion-segmenter override
hooks. The base returns candidates identical to YOLO hybrid for fine-tuning dataset reuse.
Handoff indices select which candidates -> clips -> AI analysis -> play_segments.",
    "code_ref": "backend/pipeline/segmenters/trajectory_hybrid.py:127-135
(_select_for_handoff), 113-125 (_enrich_candidate_stats), 276-285 (persist candidates), 293
(handoff gate)",
    "kind": "reuse",
    "reuse_status": "exists"
},
{
    "topic": "Detection Params Metadata (For Dataset + Fine-Tuning Traceability)",
    "detail": "trajectory_hybrid.py:218-224 ? detection_params dict (detector name,
velocity_thresh, merge_gap, runner-specific extras like detector_impl, weights_path) persisted
into candidate_segments.detection_params (JSON). Enables post-hoc analysis of which detector
settings produced which candidates. Identical structure across local vs cloud.",

```

```

        "code_ref": "backend/pipeline/segmenters/trajectory_hybrid.py:218-224 (detection_params
build), 280-284 (persist to DB)",
        "kind": "reuse",
        "reuse_status": "exists"
    },
    {
        "topic": "Chunk Overlap Deduplication in Stitcher",
        "detail": "VideoCompiler._merge_overlapping(segments) ? dedupes overlapping or abutting
(start, end) ranges before stitching. Handles edge case: two detector runs on same video (e.g.,
re-run without prune) or chunk-overlap duplicates near boundaries. Same footage never replayed.
Deterministic: sort, merge logic is pure.",
        "code_ref": "backend/pipeline/stitcher/compiler.py:81-105 (_merge_overlapping)",
        "kind": "reuse",
        "reuse_status": "exists"
    },
    {
        "topic": "Watermark Reuse Across All Clips (No Re-Bake)",
        "detail": "VideoCompiler._watermark_filter(tmp_dir) computed ONCE per
compile_highlights(), applied to every clip in _trim_one_segment(). drawtext textfile written
once, same filtergraph string used for all. If watermark encode fails, retry WITHOUT it
(branding config never breaks reel). concat stage stays -c copy because mark is baked
identically into every clip.",
        "code_ref": "backend/pipeline/stitcher/compiler.py:125-188 (_watermark_filter), 344
(build once), 259-287 (_trim_one_segment watermark attempt + retry)",
        "kind": "reuse",
        "reuse_status": "exists"
    },
    {
        "topic": "Config-Driven Detector Selection (Abstract Deployment Mode)",
        "detail": "Segmenter instantiated via segmenter_registry.get(name) + db + config. Config
structure: indexing.detector_impl ('stub', 'native', or 'auto'), indexing.{wasb,tracknet,stub}.*
for runner-specific settings. No code change needed to swap Windows<->Linux<->Cloud: config +
env vars (RALLY_DETECTOR_IMPL, WASB_WEIGHTS, WASB_PYTHON, WASB_DEVICE, etc.) determine runtime
behaviour. Same binary runs all three modes.",
        "code_ref": "backend/pipeline/segmenters/trajectory_hybrid.py:65-86 (_resolve_runner,
env override + config fallback),
backend/pipeline/detectors/{wasb_runner,native_wasb_runner,stub_runner}.py:from_indexing_cfg()",
        "kind": "config-seam",
        "reuse_status": "exists"
    },
    {
        "topic": "MISSING: Configurable Output Base Directory (Not Parameter)",
        "detail": "Hardcoded 'output/' prefix appears in trajectory_hybrid.py (lines 237, 313),
yolo_hybrid.py (line 407), gemini.py (lines 23, 119). These should be configurable
(config.output_base or from env) so cloud deployments can stage to a mounted volume or object
store. Currently, CWD must be writable and contain an 'output/' subdir ? incompatible with
containerized / read-only root deployments.",
        "code_ref": "backend/pipeline/segmenters/trajectory_hybrid.py:237, 313;
yolo_hybrid.py:407; gemini.py:23, 119",
        "kind": "config-seam",
        "reuse_status": "new"
    },
    {
        "topic": "MISSING: Configurable Temporary Staging for Large Clips (GPU Memory + Disk)",
        "detail": "NativeWasbRunner respects WASB_SCRATCH / config scratch_dir, but the
trajectory_hybrid segmenter doesn't expose a way to override the handoff clip staging dir
('output/chunks_{provider}/'). On a cloud box with limited /tmp or CWD not on a fast NVMe,
staging large clips can bottleneck. Should route through a configurable scratch base.",
        "code_ref": "backend/pipeline/segmenters/trajectory_hybrid.py:313 (hardcoded
handoff_dir), backend/pipeline/detectors/native_wasb_runner.py:179 (configurable on runner side
only)",
        "kind": "config-seam",
        "reuse_status": "partial"
    },
    {

```

```

    "topic": "MISSING: Provider Concurrency Tuning for Cloud SaaS",
    "detail": "trajectory_hybrid.py:214 ? ai_max_workers defaults to 5 (ThreadPoolExecutor).
On a cloud SaaS where one worker box may handle multiple concurrent jobs, or a provider is rate-
limited, no global rate limiter or backoff strategy exists. Concurrency is per-job, not per-box.
Future: config-driven pool size + exponential backoff for 429/timeout from Gemini.",
    "code_ref": "backend/pipeline/segmenters/trajectory_hybrid.py:214 (ai_max_workers
config), 343-347 (ThreadPoolExecutor submit)",
    "kind": "config-seam",
    "reuse_status": "new"
}
],
"config_or_env_knobs": [
    "RALLY_DETECTOR_IMPL: {stub, native, auto} ? selects CPU mock, Linux-native GPU, or WSL
(default). Overrides indexing.detector_impl.",
    "WASB_WEIGHTS, WASB_REPO_DIR, WASB_PYTHON, WASB_DEVICE, WASB_SCRATCH: NativeWasbRunner
environment overrides for Linux GPU box. Fallback to indexing.wasb.* config.",
    "GEMINI_API_KEY (or provider-specific key): Required for AI handoff; env var lookup.",
    "indexing.detector_impl: {stub, native, auto} ? persistent config choice (overridden by
env var).",
    "indexing.wasb.*, indexing.tracknet.*, indexing.stub.*: Runner-specific config (weights
path, repo dir, conda env, WSL distro, stage dir, etc.).",
    "indexing.skip_ai_handoff: bool ? if True, candidate windows become rallies directly (no
API key, no upload).",
    "indexing.ai_provider, indexing.ai_model: Provider + model name for AI handoff (default
Gemini).",
    "stitching.watermark.{enabled, text, logo_path, font_path, ...}: Watermark config for
compile_highlights().",
    "indexing.{velocity_threshold, merge_gap, min_action_window_duration, max_window_duration,
ai_handoff_padding, ai_max_workers}: Windowing + handoff tuning parameters.",
    "indexing.wasb.stream_video (WSL), indexing.wasb.stream_video (native): Fast frame-
streaming path (bit-identical output, perf win).",
],
"notes": "\nCORE CONTRACTS (Behaviour-Identical Across Modes):\n\n1. **Detection (GPU
Compute)**: DetectorRunner.run_predict() -> CSV. Three interchangeable concrete implementations
(Wasb/WSL, NativeWasb/Linux, Stub/CPU) all produce identical Frame,Visibility,X,Y schema. The
trajectory is the stable contract.\n\n2. **Windowing (CPU Pure Function)**:
TrackNetRunner.trajectory_to_action_windows() is a @staticmethod shared by ALL runners.
Deterministic, no GPU, no IO ? same trajectory + same config always yields identical candidate
windows. Used directly in tests + regression gates.\n\n3. **Stitching (FFmpeg + Config)**:
VideoCompiler.compile_highlights() is pure filesystem + subprocess. Deterministic given same
input video + segment ranges + watermark config. No hardware assumptions; identical output on
any box with ffmpeg.\n\nDEPLOYMENT MODE ABSTRACTION:\n\nThe single seam is
TrajectoryHybridSegmenter._resolve_runner(idx_cfg):\n- Reads RALLY_DETECTOR_IMPL env or
indexing.detector_impl config.\n- Returns (DetectorRunner, extras) where extras =
{detector_impl, weights_path, ...} threaded into detection_params.\n- The ONLY place mode is
chosen; process_video() is agnostic.\n- Local (Windows) default: WasbRunner (WSL + conda).\n-
Local (Linux) fast path: NativeWasbRunner (native GPU, no WSL).\n- Regression testing:
StubDetectorRunner (CPU mock, deterministic, cross-platform).\n- Production cloud:
NativeWasbRunner or Stub (depending on box config).\n\nCOUPLING POINTS (Environment/Deployment-
Specific):\n\n1. **Relative Output Paths**: 'output/', 'output/chunks_*/' hardcoded in
segmenters. On cloud, CWD != repo root ? paths break. Requires fixing to config-driven base or
env var.\n\n2. **WSL Path Translation**: /mnt/c/ only valid on Windows+WSL. NativeWasbRunner
uses native absolute paths. No abstraction; correct runner is chosen per
_resolve_runner().\n\n3. **GPU/CUDA Availability**: WasbRunner + NativeWasbRunner both assume
NVIDIA GPU (or fail healthcheck). StubRunner is CPU-only. Config device=cuda|mps|cpu, env
WASB_DEVICE override.\n\n4. **FFmpeg Binary**: All compute (inference staging, chunk extraction,
stitching) invokes subprocess ffmpeg/ffprobe. Must be on PATH. No version constraint; assumes
compatible (e.g., drawtext, concat demuxer).\n\n5. **Temporary & Scratch Space**: WasbRunner
stages ~clips/ in WSL. NativeWasbRunner uses WASB_SCRATCH or fallback output_dir. VideoCompiler
uses output_dir. Large clips (>1GB) can exhaust /tmp or slow disk. No cloud-aware staging.\n\n6.
**Watermark Font**: Bundled Apache-2.0 font path is relative (parents[2]/'assets'/...), OK for
repo-relative deployments. System font fallback ('Sans') is environment-dependent. Config
overridable.\n\n7. **Provider Rate Limiting**: AI handoff uses ThreadPoolExecutor(max_workers=5)
per job, no global backoff or provider-aware concurrency tuning. Cloud SaaS may need config-
driven pool + retry logic.\n\nPURE FUNCTIONS (Reusable, Testable):\n\n-

```

trajectory_to_action_windows(): Pure windowing. Tested in test_trajectory_runner.py without GPU. Used in yolo_hybrid + eval paths.\n- _merge_overlapping(): Pure deduplication of segment ranges. Safe to call multiple times.\n- _watermark_filter(): Builds ffmpeg filtergraph string (pure, no side effects).\n- _parse_time(): Timestamp parsing (float, int, MM:SS formats) ? pure.\n\nMISSING KNOBS FOR CLOUD DEPLOYMENT:\n\n1. Configurable output base directory (env OUTPUT_BASE or config setting).\n2. Configurable scratch/staging directory for large intermediate clips.\n3. Provider concurrency + backoff strategy (per-box rate limiting, not per-job).\n4. Metrics/observability for GPU utilization, inference latency, provider queueing.\n\nThe engine CORE (detection + windowing + stitching) is mathematically identical across modes; the gaps are environmental (paths, GPU visibility, provider access, disk space).\n"

```

    },
    {
        "area": "deployment-profile-seams",
        "findings": [
            {
                "topic": "INPUT SOURCE SELECTION",
                "detail": "Today: hardcoded to local filesystem paths (video_path argument to /api/process or CLI). The worker module supports URI-based fetch via storage.fetch(args.video_uri) with parsing (local/s3/gcs/gdrive://) but the main CLI has no input-source abstraction. Selection path: backend/config/models.py StorageConfig (backend, output_root, workspace_root) only controls OUTPUT; INPUT source comes from ProcessRequest.video_path (plain string, local-only). The new deployment profile must add: (1) storage.input_root or input_backend selection, (2) input_source enum (local-path|usb-mount|gdrive-mount|s3-bucket|gcs-bucket), (3) resolution precedence for the input root at startup.",
                "code_ref": "backend/main.py:254-265 (ProcessRequest.video_path hardcoded), backend/worker/__main__.py:98 (worker uses storage.fetch pattern), backend/config/models.py:295-320 (StorageConfig covers output only)",
                "kind": "coupling-point",
                "reuse_status": "partial"
            },
            {
                "topic": "COMPUTE BACKEND SELECTION",
                "detail": "Today: detector_impl env var (RALLY_DETECTOR_IMPL) or indexing.detector_impl selects GPU runner type (auto=WSL, native=Linux-GPU-box, stub=CPU-mock). This is PARTIALLY exposed: (1) trajectory hybrids call _resolve_runner() which checks indexing.detector_impl, (2) skip_ai_handoff boolean selects local-noai vs cloud-provider path. What's missing: (a) no top-level 'compute_backend' enum that picks between {local-GPU|Cloud-Run|stub}, (b) no orchestration of detector_impl + skip_ai_handoff + storage.backend together, (c) no startup config that declares 'this deployment runs on Google Cloud / locally'. The new profile must define: compute_target enum (local_gpu|cloud_run|cpu_mock) that sets detector_impl + ai_provider + workspace_root coherently.",
                "code_ref": "backend/config/models.py:125-206 (IndexingConfig.detector_impl 'auto'|'stub'|'native'), backend/pipeline/segmenters/trajectory_hybrid.py:65-86 (_resolve_runner precedence), backend/pipeline/segmenters/trajectory_hybrid.py:158 (skip_ai_handoff boolean)",
                "kind": "core-contract",
                "reuse_status": "partial"
            },
            {
                "topic": "STORAGE BACKEND ORCHESTRATION",
                "detail": "Exists: backend/storage/ module with 4 backends (local/s3/gcs/gdrive) selected by StorageRef.parse_uri(). Config: backend/config/models.py StorageConfig.backend + per-backend dicts (local/s3/gcs/gdrive with secrets-free endpoint/region/mount_root). SEAM: storage.backend picks the OUTPUT backend, but INPUT is unselected (always local). The worker (backend/worker/__main__.py) uses storage.fetch/put/list_uris correctly with config.storage.model_dump() passed through. What a deployment profile needs: (1) separate input_backend vs output_backend, (2) workspace_root URI (already exists, default-empty falls back to output.output_dir), (3) per-video workspace subfolders (backend/utils/workspace.py exists, gated by storage.single_folder_per_video flag?DEFAULT OFF per PER_VIDEO_WORKSPACE.md)",
                "code_ref": "backend/storage/__init__.py:44-76 (get_storage entry point), backend/config/models.py:295-320 (StorageConfig), backend/utils/workspace.py:46-115 (workspace helper, pure string joins), backend/worker/__main__.py:88-159 (cmd_infer uses storage.fetch + storage.put pattern)",
                "kind": "config-seam",

```

```

    "reuse_status": "exists"
  },
  {
    "topic": "AI PROVIDER HANDOFF TOGGLE",
    "detail": "Exists as indexing.skip_ai_handoff boolean (default False). When True:
trajectory candidate windows become rallies directly, no AI upload, no API key needed. When
False: windows sent to ai_provider (config says 'gemini'; see backend/config/models.py
AppConfig.ai_provider='gemini') for semantic validation. This is the LOCAL vs CLOUD-SERVING
lever. A deployment profile should expose: skip_ai_handoff as a top-level mode (fully_local_mode
or cloud_serving_mode) and lock it based on compute_target.",
    "code_ref": "backend/config/models.py:330 (ai_provider='gemini'),
backend/config/models.py:195-200 (skip_ai_handoff default False),
backend/pipeline/segmenters/trajectory_hybrid.py:158-311 (skip_ai logic + Phase 3 handoff)",
    "kind": "config-seam",
    "reuse_status": "exists"
  },
  {
    "topic": "PER-VIDEO WORKSPACE & ARTIFACT LAYOUT",
    "detail": "NEW: backend/utils/workspace.py helpers +
storage.{workspace_root,single_folder_per_video,workspace_prefix} config fields. Currently gated
by single_folder_per_video=False (legacy flat layout untouched per PER_VIDEO_WORKSPACE.md D5).
Layout: <workspace_root>/[<prefix>]/<video_id>/{source,detector,outputs,reels,logs,tmp}/. Helper
is pure (no I/O); write sites not yet converged (P2-P4 in progress). The deployment profile
must: (1) set workspace_root URI, (2) optionally set single_folder_per_video=True for cloud path
(will be gated-flipped by Launch Report), (3) configure workspace_prefix for cloud namespace
tidiness.",
    "code_ref": "backend/utils/workspace.py:46-115 (pure path helper),
backend/config/models.py:306-315 (workspace_root, single_folder_per_video, workspace_prefix),
docs/PER_VIDEO_WORKSPACE.md §3-4 (design + reuse map)",
    "kind": "net-new-gap",
    "reuse_status": "partial"
  },
  {
    "topic": "CONFIG LOADING & PRECEDENCE",
    "detail": "Current: load_config(path) reads config.json ? Pydantic AppConfig with built-
in defaults. No environment-based auto-selection. Precedence: built-in defaults ? config.json
file ? (future env/CLI overrides per loader.py:58 comment). The deployment profile needs: (1) a
'profile' or 'deployment_mode' top-level field (or env DEPLOYMENT_PROFILE), (2) preset bundles
(local-all, cloud-serving-gcp, cloud-serving-do) that lock coherent sets of: storage.backend,
workspace_root, detector_impl, skip_ai_handoff, ai_provider, (3) per-profile startup
checks/warnings (e.g. if Cloud-Run mode but no GCS creds, fail fast).",
    "code_ref": "backend/config/loader.py:53-97 (load_config + precedence),
backend/config/models.py:322-381 (AppConfig root + model_validate), backend/main.py:85-87
(legacy thin wrapper load_config)",
    "kind": "net-new-gap",
    "reuse_status": "new"
  },
  {
    "topic": "STARTUP ENVIRONMENT DETECTION",
    "detail": "Missing: no detection of deployment environment (local machine, Cloud Run
container, GPU box, CI). The HOSTING_PLAN.md mentions Cloud Run (K_SERVICE, workload identity)
but the code has no check for it. Today, environment is implicit: the user runs `python -m
backend.main` on their laptop, or a Cloud Run YAML points to the same code. A profile system
should auto-detect or require explicit declaration: is_cloud_run (K_SERVICE env), is_gpu_box
(torch.cuda.is_available or a config flag), is_ci (CI=true env), etc., so preset profiles can
warn early if requirements aren't met.",
    "code_ref": "backend/main.py:1-35 (no startup checks),
backend/worker/__main__.py:385-391 (same for worker CLI)",
    "kind": "net-new-gap",
    "reuse_status": "new"
  },
  {
    "topic": "KNOBS THAT ALREADY EXIST (REUSE)",
    "detail": "Deployment profile can be built from these existing, coherent knobs: (1)
storage.backend ('local'|'s3'|'gcs'|'gdrive'), (2) storage.workspace_root (URI), (3)

```

```

storage.single_folder_per_video (bool, default False), (4) storage.workspace_prefix (str,
default 'workspaces'), (5) indexing.detector_impl ('auto'|'stub'|'native'), (6)
indexing.skip_ai_handoff (bool, default False), (7) ai_provider (str, 'gemini'), (8)
security.allowed_video_root (for input validation in hosted mode). These are already wired end-
to-end in the worker (cmd_infer/cmd_train) and segmenters.",
    "code_ref": "backend/config/models.py (entire file), backend/storage/__init__.py:44-76,
backend/utls/workspace.py, backend/pipeline/segmenters/trajectory_hybrid.py:65-86",
    "kind": "reuse",
    "reuse_status": "exists"
},
{
    "topic": "NET-NEW PROFILE CONFIG STRUCTURE",
    "detail": "Proposed additions to AppConfig: (1) Top-level 'deployment' section with
profile='local-all'|'cloud-serving'|'stub' (or leave empty for manual config); (2) auto-
detection helpers in load_config() that set a DEFAULT profile based on env (DEPLOYMENT_PROFILE
env var, K_SERVICE, torch availability); (3) profile preset bundles (json presets or code switch
on profile name) that lock coordinated values. Example: profile='cloud-serving' ? backend='gcs',
workspace_root='gs://...' (from env/secret manager), detector_impl='native' (if GPU box env
detected), skip_ai_handoff=False. All existing knobs remain individually overridable
(config.json or --set arg).",
    "code_ref": "backend/config/models.py (extend AppConfig + add DeploymentConfig section),
backend/config/loader.py (extend load_config with profile auto-detect + preset application)",
    "kind": "net-new-gap",
    "reuse_status": "new"
},
{
    "topic": "WORKER (BATCH ENTRYPOINT) INTEGRATION",
    "detail": "Worker cmd_infer (backend/worker/__main__.py:88-159) already fully supports
the storage seam: --video-uri (with parse_uri), --out-uri, --config, --set k=v. It composes
storage.fetch + segmenter + storage.put correctly. For a deployment profile: the worker's --set
flag can apply profile-level overrides (e.g. `--set storage.backend=gcs --set
indexing.detector_impl=native`), or the profile config.json file is passed via --config. No new
wiring needed; the worker is already profile-agnostic.",
    "code_ref": "backend/worker/__main__.py:88-159 (cmd_infer),
backend/worker/__main__.py:49-59 (_apply_overrides for --set),
backend/worker/__main__.py:347-382 (arg parser exposes --config, --set)",
    "kind": "reuse",
    "reuse_status": "exists"
}
],
"config_or_env_knobs": [
    "storage.backend (local|s3|gcs|gdrive) ? output backend selection",
    "storage.output_root ? override for artifact root (falls back to output.output_dir)",
    "storage.workspace_root ? durable workspace URI for per-video folders",
    "storage.workspace_prefix ? cloud namespace prefix (default 'workspaces')",
    "storage.single_folder_per_video ? enable per-video workspace layout (default False)",
    "storage.local.base_dir ? local storage base (unused if absolute paths)",
    "storage.s3.endpoint_url, region, addressing_style ? S3/R2/D0 Spaces config",
    "storage.gcs.project ? GCP project for signed URLs",
    "storage.gdrive.mount_root ? Google Drive mount path (auto-detect if unset)",
    "indexing.detector_impl (auto|stub|native) ? GPU runner selection",
    "indexing.skip_ai_handoff ? toggle local-only vs cloud AI path",
    "indexing.default_segmenter ? which segmenter to run",
    "ai_provider ? AI backend ('gemini')",
    "ai_model ? AI model name",
    "output.output_dir ? local artifact root (fallback for workspace_root)",
    "security.allowed_video_root ? input path whitelist for hosted mode",
    "[NEW] deployment.profile ? preset bundle (local-all|cloud-serving|stub)",
    "[NEW] deployment.environment ? explicit hint (local-gpu|cloud-run|cpu-mock) for startup
checks"
],
"notes": "A single deployment profile / startup config must orchestrate three coupled seams:
(1) INPUT-SOURCE: add input_backend selection (parallel to output storage.backend) + input_root
(parallel to workspace_root) so the CLI can pull from local/usb/gdrive/bucket; (2) COMPUTE:
define compute_target enum {local-gpu|cloud-run|cpu-mock} that locks detector_impl +

```

skip_ai_handoff + workspace strategy coherently (local-gpu runs native WASB + full AI; cloud-run runs stub detector + no AI; cpu-mock runs stub for regression tests); (3) STORAGE: the backend/utils/workspace.py helper + single_folder_per_video flag already exist (gated-OFF per PER_VIDEO_WORKSPACE.md), and backend/storage/ supports 4 backends via URI dispatch. No core rework needed ? compose and gate existing knobs. The worker (cmd_infer/cmd_train) is already profile-agnostic and accepts --config + --set overrides. Knobs that are NEW: a top-level 'deployment' config section (profile name + optional auto-detect) and env-var-driven startup checks (K_SERVICE for Cloud Run, torch.cuda for GPU box). The main CLI lacks input-source abstraction (always assumes local paths) ? that's the biggest gap. A profile system should add an input_backend + input_root parallel to the existing output-side StorageConfig.</notes>\n"

```

    },
    {
        "area": "Control plane + hosted-service surface mapping: FastAPI API + job queue + cost tracking gaps",
        "findings": [
            {
                "topic": "Core hosting surface ? ALREADY EXISTS",
                "detail": "FastAPI endpoints: GET /api/health (liveness), POST /api/process (submit segmentation job), GET /api/jobs/{job_id} (poll status), POST /api/query (segment filtering), POST /api/compile (stitch highlights), GET /api/highlights/{video_id}/{filename} (download reel). Chunked upload (POST /api/upload/init, PUT /api/upload/{id}/part/{ix}, POST /api/upload/{id}/complete, DELETE /api/upload/{id}) for browser ? server assembly (?95 MB parts, 4 GB max file). Job worker (async ThreadPoolExecutor max_workers=1) drains jobs table via claim_due_job(), runs _execute_processing() (validate ? segment ? report), records result/error/progress/video_id to job row.",
                "code_ref": "backend/main.py:36-414 (FastAPI app, endpoints), backend/api/uploads.py (chunked upload router), backend/api/job_worker.py:100-122 (_drain_due_jobs loop), backend/infrastructure/database.py:486-627 (jobs table CRUD + claim_due_job atomic CAS)",
                "kind": "core-contract",
                "reuse_status": "exists"
            },
            {
                "topic": "Jobs table schema ? FOUNDATION IN PLACE",
                "detail": "v2 migration creates 'jobs' with: id (PK), video_path, video_id, segmenter, player_pool, max_frames, status (queued|running|done|failed), progress, error, result (full response JSON), created_at, started_at, finished_at. v3 adds policy (JSON ExecutionPolicy) + next_poll_at (self-scheduling). v4 adds nullable user_id (tenant scoping, currently NULL = local). Indexes: idx_jobs_status (status), idx_jobs_due (status, next_poll_at). See backend/infrastructure/database.py:180-217 (_migrate_to_v2/v3/v4).",
                "code_ref": "backend/infrastructure/database.py:180-217, 538-627",
                "kind": "core-contract",
                "reuse_status": "exists"
            },
            {
                "topic": "Cost ledger columns ? NET-NEW GAP (identified in COST-ATTRIBUTION.md)",
                "detail": "jobs table has NO cost/billing columns. Per-video inference billing requires: owner_id TEXT (tenant for multi-user), compute_backend TEXT (runpod_serverless|gcp_cloudrun|do_droplet?), gpu_type TEXT (L4|T4|A4000?), gpu_active_seconds REAL, wall_seconds REAL, bytes_out INTEGER (egress), gemini_cost_usd REAL, pricebook_version TEXT, cost_usd REAL (total), cost_breakdown_json TEXT ({gpu: .., egress: .., api: ..}). Also new tables: pricebook (version, resource, gpu_type, unit_price_usd, currency, effective_at) and training_contributions (training_job_id, video_id, weight). See docs/COST-ATTRIBUTION.md §3.",
                "code_ref": "docs/archives/past_projects/DECOUPLED_COMPUTE/05-COST-ATTRIBUTION.md:36-61",
                "kind": "net-new-gap",
                "reuse_status": "new"
            },
            {
                "topic": "Auth gap ? CORS '*' + no identity verification",
                "detail": "CORS configured with allow_origins=['*'], allow_credentials=False (per backend/main.py:41-47 comment: 'fetch spec rejects allow_origins=\"*\" with allow_credentials=True'). No endpoint requires identity; health/process/jobs/query/compile all have open access. Path validation (security.allowed_video_root) is advisory, not enforced at auth layer. HOSTING_PLAN.md §2 specifies: alpha uses Cloudflare Access (email-OTP allowlist, free ?50 users) + backend trusts Cf-Access-Authenticated-User-Email header; beta uses Firebase

```

```

Auth. Multi-tenant scoping requires owner_id extraction + per-user query filters (not yet
wired).",
    "code_ref": "backend/main.py:41-47 (CORS), 263 (allowed_root check only),
backend/config/models.py:278-292 (SecurityConfig.allowed_video_root)",
    "kind": "config-seam",
    "reuse_status": "partial"
},
{
    "topic": "Cloud Run + GPU dispatch ? NET-NEW (not in codebase)",
    "detail": "Currently segmentation (validation ? segment ? finalize) runs in-process on
the FastAPI worker thread (single ThreadPoolExecutor). HOSTING_PLAN.md §3 (beta) describes Cloud
Run control plane dispatching jobs to a 'byo_worker' GPU box that pulls from GCS.
DECOUPLED_COMPUTE/02 identifies WSL/GPU portability risk (HIGH for TrackNet/WASB, LOW for
gemini/motion) and the need for LinuxNativeRunner parity test on M3.5 Linux GPU. No dispatch
code exists: need Cloud Task / Cloud Pub/Sub / simple HTTP poll contract to push jobs from Cloud
Run control plane to GPU worker(s), pull results back. Execution must meter gpu_active_seconds +
wall_seconds for cost attribution.",
    "code_ref": "backend/main.py:139-241 (_execute_processing, synchronous),
docs/HOSTING_PLAN.md:50-61 (beta topology), docs/DECOUPLED_COMPUTE/02:7-26 (compute
portability)",
    "kind": "net-new-gap",
    "reuse_status": "new"
},
{
    "topic": "Per-video cost metering ? NET-NEW (provider metrics exist, aggregation does
not)",
    "detail": "Gemini provider (backend/providers/gemini.py) already returns metrics dict
from analyze_video(): {upload_time, process_time, gen_time, oversize_skipped}. Cost-attribution
design specifies: CPU-metered jobs (TrackNet/WASB on Linux GPU box) record gpu_active_seconds
via timestamp probes around GPU code; serverless (Cloud Run, RunPod) pulls exact gpu_seconds
from platform per-invocation report (COST-ATTRIBUTION.md §2 table). Egress metered from
bytes_downloaded (R2 zero egress). No aggregation logic yet: need to collect metrics from
segmenter (including GPU time if on-box GPU used), provider (API cost), and stitcher (ffmpeg CPU
time, bytes_out), sum into cost_usd + cost_breakdown_json on the job row.",
    "code_ref": "backend/providers/gemini.py:38-100 (analyze_video metrics), docs/COST-
ATTRIBUTION.md:23-34 (metering sources)",
    "kind": "net-new-gap",
    "reuse_status": "new"
},
{
    "topic": "Segment + stitch dispatch separation ? NET-NEW",
    "detail": "Currently /api/compile runs VideoCompiler.compile_highlights()
(backend/pipeline/stitcher/compiler.py, ffmpeg-based) synchronously in the request handler. For
cloud topologies, stitching should either: (a) stay in Cloud Run (CPU only, low cost, 3 min per
reel) or (b) move to a queue (if users submit dozens per video). Segmentation is the GPU-heavy
critical path. Schema supports both: job.segmenter specifies the segment plugin; the compile
endpoint is independent. Dispatch design must clarify: segment = GPU job (dispatch to worker),
stitch = Cloud Run synchronous (cheaper) OR queued job (if bursty).",
    "code_ref": "backend/main.py:356-413 (POST /api/compile),
backend/pipeline/stitcher/compiler.py (VideoCompiler.compile_highlights)",
    "kind": "net-new-gap",
    "reuse_status": "new"
},
{
    "topic": "Execution policy + resumable jobs ? FOUNDATION IN PLACE",
    "detail": "backend/infrastructure/execution_policy.py defines ExecutionPolicy (mode:
WAIT_AND_RESUME|ABORT|BLOCK, poll_every_n_sec, op_timeout_sec, max_wait_sec, max_retries,
backoff_base_sec, on_timeout, resumable). run_bounded() wraps long ops (e.g., Gemini
upload/process) with per-attempt timeout + exponential backoff. poll_until() polled a promise.
JobWaiting exception (backend/api/job_worker.py:41-56) signals a job to reschedule: worker
catches it, persists progress + next_poll_at, releases thread. claim_due_job() re-claims waiting
jobs when due. This enables long Gemini waits (20 min cap) + multi-attempt robustness without
blocking. For Cloud Run + GPU dispatch: same JobWaiting seam reschedules a job waiting for a GPU
worker result.",
    "code_ref": "backend/infrastructure/execution_policy.py (ExecutionPolicy, run_bounded,

```

```

poll_until), backend/api/job_worker.py:41-94 (JobWaiting exception + set_job_waiting call)",
    "kind": "core-contract",
    "reuse_status": "exists"
},
{
    "topic": "Config knobs for cloud dispatch ? PARTIAL (foundation exists, cloud params
missing)",
    "detail": "AppConfig (backend/config/models.py) already typed:
security.allowed_video_root (path validation), security.upload_dir/max_upload_mb/max_part_mb
(upload limits), storage backend (local|s3|gcs|gdrive, not yet wired). IndexingConfig has
skip_ai_handoff (fully-local mode). Missing: compute_backend selector
(runpod_serverless|gcp_cloudrun|do_droplet?), gpu_type options, per-backend
credentials/endpoints (Cloud Run project, RunPod API key, DO Droplet IP), target_worker_urls
(polling endpoints on GPU box or serverless). These should extend AppConfig as a new 'compute'
section: {backend, gpu_type, worker_urls, runpod_api_key, gcp_project, ?}.",
    "code_ref": "backend/config/models.py:278-320 (SecurityConfig, StorageConfig),
HOSTING_PLAN.md:50-61 (GCP+DO topology)",
    "kind": "config-seam",
    "reuse_status": "partial"
},
{
    "topic": "Tensor/I/O adapter for cloud GPU box ? NET-NEW",
    "detail": "Local-file assumptions throughout: cv2.VideoCapture, ffmpeg -i <path>, whole-
file MD5 (compute_video_id), allowed_video_root path validation. DECOUPLED_COMPUTE/02 §4
identifies this as a blocker for cloud GPU. Need a `fetch(uri)?local` + `put(local?uri)`
adapter: on a GPU box pulling from GCS bucket, videos are staged to local /scratch, processed,
results (highlights.mp4, report.json, intervals.csv) uploaded back. On Cloud Run (no GPU), same
adapter serves Gemini-path Uploads. Gemini provider already handles HTTP upload/download
(backend/providers/gemini.py). Missing: GCS (or R2 / S3) client for bucket fetch/put + lifecycle
management (delete intermediate artifacts after X days).",
    "code_ref": "backend/pipeline/segmenters/trajectory_hybrid.py:196,272 (hardcoded
'output'), backend/main.py:263 (allowed_video_root check), DECOUPLED_COMPUTE/02:27-30 (Path/I/O
blockers)",
    "kind": "net-new-gap",
    "reuse_status": "new"
},
{
    "topic": "Per-tenant scoping + multi-user isolation ? PARTIAL (schema seam in place,
query filters missing)",
    "detail": "v4 migration adds nullable user_id to videos + candidate_segments (NULL =
local install = backward-compatible). Partial-unique indexes on user_id keep the NULL fast path.
Database has no query filters on user_id yet. job.owner_id (proposed in COST-ATTRIBUTION.md)
scopes billing. HOSTING_PLAN.md §2 (alpha) uses Cloudflare Access: backend must extract Cf-
Access-Authenticated-User-Email header and scope /api/process, /api/query, /api/compile,
/api/jobs/{id} to that user. Multi-tenant reads (list my videos, my jobs, cost history) need to
be wired into the endpoints and indexed for performance.",
    "code_ref": "backend/infrastructure/database.py:219-237 (v4 migration: user_id columns +
partial indexes), backend/main.py:263-285 (POST /api/process has no user scoping),
docs/HOSTING_PLAN.md:42 (Cf-Access header)",
    "kind": "coupling-point",
    "reuse_status": "partial"
},
{
    "topic": "Cost endpoint contract ? NET-NEW",
    "detail": "COST-ATTRIBUTION.md §4-5 proposes: GET /api/jobs/{id}/cost (returns cost_usd
+ cost_breakdown_json), GET /api/videos/{video_id}/cost (per-video cost history, owner-scoped).
No endpoints exist yet. Schema delta is designed but not applied. Cost computation (aggregating
gpu_seconds + egress + api calls into cost_usd via pricebook lookup) is not implemented. Needs:
(a) v6 migration to add cost columns, (b) helper functions to compute cost from metrics +
pricebook, (c) endpoints to expose cost history, (d) pricebook seeding (hardcoded initial rates
or loaded from config).",
    "code_ref": "docs/COST-ATTRIBUTION.md:36-51 (schema delta), 70-77 (endpoint design)",
    "kind": "net-new-gap",
    "reuse_status": "new"
}

```

```

],
"config_or_env_knobs": [
  "security.allowed_video_root (path validation, advisory only)",
  "security.upload_dir (staging zone for browser uploads)",
  "security.max_upload_mb / security.max_part_mb (100 MB body limit from free-tier proxy)",
  "storage.backend (local|s3|gcs|gdrive, foundation laid, not fully wired)",
  "indexing.skip_ai_handoff (local-only mode, skips Gemini)",
  "indexing.ai_chunk_scale_width (re-encode resolution before cloud upload)",
  "stitching.watermark (branding config for compiled reels)",
  "net-new-needed: compute.backend (runpod_serverless|gcp_cloudrun|do_droplet|local_box)",
  "net-new-needed: compute.gpu_type (L4|T4|A4000|?)",
  "net-new-needed: compute.worker_urls (polling targets for GPU jobs)",
  "net-new-needed: cost.pricebook_version (versioned rates for billing)"
],
"notes": "***WHAT EXISTS:** The hosted-service skeleton is in place ? FastAPI endpoints
(/api/process, /api/jobs/{id}, /api/compile, /api/highlights), chunked upload (B2), async job
worker (ThreadPoolExecutor + claim_due_job), job state machine
(queued?running?done/failed/waiting), and execution policy (bounded waits, exponential backoff,
resumable jobs). **THE GAPS:** (1) **AUTH:** CORS '*' + no identity verification; Cloudflare
Access header (Cf-Access-Authenticated-User-Email) is documented but not extracted/verified. (2)
**DISPATCH:** No Cloud Run / GPU worker dispatch ? segmentation runs in-process. No job
push/pull contract for remote compute. (3) **COST LEDGER:** jobs table has NO cost columns
(owner_id, compute_backend, gpu_type, gpu_active_seconds, wall_seconds, bytes_out,
gemini_cost_usd, cost_usd, cost_breakdown_json). pricebook table not created. Cost aggregation
logic absent. (4) **CONFIG:** Missing compute.backend / compute.gpu_type / compute.worker_urls
knobs. (5) **I/O ADAPTER:** No GCS/S3/R2 fetch?local?put for cloud GPU box. Videos assume local
file. (6) **TENANT SCOPING:** user_id columns exist (v4), queries don't filter on them. Job
owner_id not yet added. (7) **STITCHING:** unclear if stitch should stay in Cloud Run (sync) or
move to job queue. **REUSE POTENTIAL:** ExecutionPolicy (WAIT_AND_RESUME), run_bounded,
poll_until, and the JobWaiting seam all carry over exactly to a Cloud Run + GPU dispatch model ?
reschedule a job when waiting for GPU result, re-claim when due. Same job table + claim_due_job
CAS. Gemini provider metrics already structured (analyze_video() returns dict).
**DEPENDENCIES:** Beta topology (HOSTING_PLAN.md §3) requires GCS bucket access (browser signed
URLs, GPU box pull), Cloud Run FastAPI deployment, and a GPU worker (initially the owner's home
box, later a Linux container). COST-ATTRIBUTION.md is design-complete and ready to implement as
v6 migration + helper functions."
},
{
  "area": "Testing Surface ? Multi-Profile Strategy for Badminton Highlight Indexer",
  "findings": [
    {
      "topic": "Test Entry Point & Organization",
      "detail": "run_all_tests.py is the single discoverable entry point (passes all args to
pytest). Default quiet mode, can filter by -k/--cov, etc. Entire suite is fully mocked (no GPU,
no network, no WSL). Optional obsreport package skip-gates reporting tests via
pytest.importorskip. Exit code is pytest's own (0=green).",
      "code_ref": "run_all_tests.py",
      "kind": "core-contract",
      "reuse_status": "exists"
    },
    {
      "topic": "CI Lanes Structure",
      "detail": "Four production lanes in .github/workflows/ci.yml: (1) lint (ruff,
pyflakes/bugbear), (2) typecheck (mypy lenient baseline with explicit per-module quarantine
list), (3) import-smoke (syntax sweep + GPU-stack-free import tests), (4) full-suite (CPU-only
torch, coverage floor 70%, skips obsreport if absent). Additionally: golden-crossos matrix
(ubuntu/windows/macros) replays committed golden fixtures with parse-compare on CONTROL path only
(no numpy execution, pure stdlib for determinism).",
      "code_ref": ".github/workflows/ci.yml:1-126",
      "kind": "core-contract",
      "reuse_status": "exists"
    },
    {
      "topic": "M0 ? Golden Fixtures (M1 Synthetic Tier-0 CONTROL Regression Gate)",
      "detail": "backend/eval/golden_fixtures.py is the $0 CI regression gate: deterministic,

```

synthetic, video-free CharacterizationSeeder. Replays ONLY the CONTROL variant (pure stdlib, no numpy on replay path). Built-in scenarios: clean_single_rally, multi_rally_with_deadtime, occlusion_break. Each freeze pins windowing params verbatim (vt=0.02, mg=2.0, mwd=2.0, max_win=0 for W1-OFF). Fixtures are behavior-locking, not correctness (\$6 honest-limits caveat). Parse-compare (never byte-compare): exact ints/structure, abs-tol floats at 1e-6. Tier-0 privacy invariant enforced: no person/pose/gait/face/audio keys (FORBIDDEN_KEYS tuple). Committed slugs in eval_baselines/fixtures/manifest.json drive test parametrization.",

```
"code_ref": "backend/eval/golden_fixtures.py:1-100, tests/test_golden_fixtures.py:1-50",
"kind": "core-contract",
"reuse_status": "exists"
```

```
},
{
```

```
"topic": "M2 ? Real Fixture De-Attribution & Refusal Gate (P1)",
"detail": "golden_real_fixtures.py (split from golden_fixtures for clarity) implements P1 tooling to freeze OWNED, minors-free real data into committable Tier-0 fixtures. RefusalError gate enforces: minors_free=True, owned=True, non-blank license, strict-slug label (no free-text identifiers, no spaces/punctuation/uppercase). source_note is accepted-but-not-persisted (red-team #1b: operator free-text never reaches committed bytes). Time-origin reset is metric-invariant (score identical, segments shift toward 0). Produces fx_<hex> surrogate slugs + kind='cleared_real' provenance tags. Positive privacy assertion scans for FORBIDDEN_KEYS in expected.json and source leaks in provenance. Tests in test_golden_real_fixtures.py prove refusal blocks all unsafe combos, freezes are de-attributed, round-trip via replay, and metrics are invariant under time reset.",
```

```
"code_ref": "backend/eval/golden_real_fixtures.py,
tests/test_golden_real_fixtures.py:1-200",
"kind": "core-contract",
"reuse_status": "exists"
```

```
},
{
```

```
"topic": "M3.5 ? Stub Detector Seam (GPU-Free E2E Deterministic Path)",
"detail": "backend/pipeline/detectors/stub_runner.py: a CPU-only DetectorRunner that produces deterministic Frame,Visibility,X,Y trajectory CSV with no GPU/WSL/model weights. Dual mode: (1) synth (default, num_rallies in-flight bursts separated by dead gaps) ? stable candidate count; (2) replay (emit golden CSV verbatim). Configured via detector_impl='stub' or RALLY_DETECTOR_IMPL=stub env (env wins). Injection hook TrajectoryHybridSegmenter._resolve_runner() swaps stub in while leaving default path untouched. Makes entire trajectory-hybrid pipeline runnable and regression-gateable on CPU. Deterministic + cross-OS. Tests: runner contract (DetectorRunner ABC, healthcheck), CSV validity, synthetic trajectory ? windows via shared TrackNetRunner parse/window code, determinism, replay mode, resolve hook (default, config, env override).",
```

```
"code_ref": "backend/pipeline/detectors/stub_runner.py:1-130,
tests/test_stub_runner.py:1-130",
"kind": "coupling-point",
"reuse_status": "exists"
```

```
},
{
```

```
"topic": "Storage Backend Fakes (In-Memory S3/GCS, Local Mount for GDrive)",
"detail": "tests/test_storage.py defines _FakeS3Client (dict-backed store, upload_file/download_file/head_object/paginate/delete/presigned_url), _FakeBlob/_FakeGCSCClient (google-cloud-storage mirror, bucket.blob + list_blobs). S3/GCS backends accept injected clients; tests pass fakes (no boto3/google-cloud-storage, no network). GDrive backend reads locally MOUNTED Google Drive (Drive for Desktop); tests stand in tmp_path as the mount root (pure files, no Drive API). All backends exercise the same StorageBackend ABC (fetch_to_local, put, exists, stat, list, delete, signed_url). Tests prove roundtrip (put ? exists ? stat ? list ? fetch ? delete) and error handling (gdrive not mounted, unsynced path).",
```

```
"code_ref": "tests/test_storage.py:125-249",
"kind": "config-seam",
"reuse_status": "exists"
```

```
},
{
```

```
"topic": "Experiment Harness & Variant Profiles",
"detail": "backend/eval/experiment.py defines Variant dataclass as declarative re-scoring recipe (NOT a code branch). Core fields: name, segmenter, min_crossings (Gate A), min_players (Gate B), classifier_model (pinned JSON), feature_names (train-fresh recipe), threshold, merge_gap, tune_threshold, calibrate. Variant.is_learned() ? True iff
```

classifier_model or feature_names set. CONTROL is the blessed no-op variant (all defaults, no learned filter). Three modes: compare (labeled A/B vs golden, LOVO-honest), compare_unlabeled (SxS on new footage, divergence spot-check), promotion (run_regression.py gate, exit 0/1/3). Every knob defaults to control behavior (byte-identical guarantee for merged-but-disabled cues). VideoCandidates(name, intervals, features, gts) is the persisted-candidate substrate reused across all variants. Tests: no-regression invariant (disabled cue byte-identical to control), fusion-schema detection, golden loaders audio policy differences, LOVO honest-fold mechanics.",

```
"code_ref": "backend/eval/experiment.py:1-150, tests/test_experiment.py:1-100",
"kind": "core-contract",
"reuse_status": "exists"
},
{
  "topic": "Evaluation Stack Reuse (metrics, training, classifier, gemini_refine)",
  "detail": "backend/eval/metrics.py: score(preds, gts, iou) returns ScoreRecord with
precision/recall/f1/mean_iou/errors. backend/eval/training.py: label_candidates(vc, iou) assigns
y?{0,1} per interval using IoU matcher. backend/eval/classifier.py: LogisticRegression (JSON-
pinnable). backend/eval/gemini_refine.py: merge_overlaps(intervals, gap) post-filter.
backend/eval/regression.py: gt_hash(gts) fingerprints label set. Every experiment/variant reuses
these unchanged; no new scoring math in the variant layer. Golden fixtures + experiment tests
both invoke the same eval stack VERBATIM, proving the shared path holds under variant re-
parametrization.",
  "code_ref": "backend/eval/metrics.py, training.py, classifier.py, gemini_refine.py,
regression.py",
  "kind": "reuse",
  "reuse_status": "exists"
},
{
  "topic": "Regression Testing & Baseline Store",
  "detail": "backend/eval/regression.py: save_baseline(video_id, stage, precision, recall,
f1, iou_threshold, gt_fingerprint, path); load_baselines(path); regress(db, video_id, gts, iou,
baseline_path, tolerance=0.05) ? RegressionResult with regressed flag + reasons. Tests: no
baseline = not a regression, matching baseline passes, recall drop beyond tolerance regresses,
within tolerance passes. CLI contract: run_regression.py exit 0=green, 1=regressed, 3=missing
baseline (unless --allow-missing-baseline). Baseline store is human-readable JSON under
eval_baselines/ (tracked, not gitignored).",
  "code_ref": "backend/eval/regression.py, tests/test_eval_regression.py:1-150,
run_regression.py",
  "kind": "core-contract",
  "reuse_status": "exists"
},
{
  "topic": "Per-Video Metrics Aggregation",
  "detail": "backend/eval/batch.py: default_stages_for() selects stages per detector
(motion/gemini ? [final], yolo_hybrid ? [candidates, final]); aggregate(per_video, stages)
computes micro precision/recall/f1, macro F1, TP-weighted mean IoU, from per-video
{true_positives, false_positives, false_negatives, mean_iou, f1}. Tests: micro/macro
aggregation, zero-division handling, stage filtering.",
  "code_ref": "backend/eval/batch.py, tests/test_eval_batch.py:1-68",
  "kind": "reuse",
  "reuse_status": "exists"
},
{
  "topic": "Pytest Configuration & Collection",
  "detail": "pytest.ini sets testpaths=tests (collect only from tests/, not scratch/). No
confctest.py in repo (fixtures are inlined in test files or use pytest.fixture decorators). Tests
use tmp_path (pytest built-in), parametrization via @pytest.mark.parametrize, capsys for stdout
capture, monkeypatch for env/config injection. Database fixture (test-local SQLite via tmp_path)
for regression/eval tests.",
  "code_ref": "pytest.ini, tests/confctest patterns",
  "kind": "config-seam",
  "reuse_status": "exists"
},
{
  "topic": "Profile Strategy ? Core Proof Points",
  "detail": "Design a multi-profile test matrix to prove: (1) Core unaffected across
```

profiles: CONTROL golden fixtures pass on all platforms (golden-crossos matrix). Disabled cues are byte-identical to control (test_disabled_cues_byte_identical_to_control in experiment). (2) Cloud path works (mocked): storage fakes (S3/GCS in-memory, GDrive mount-local) prove backend abstraction holds; inject fake clients, roundtrip, verify no real SDK calls. (3) GPU-free E2E: stub detector seam proves trajectory?candidates?windows via shared model-agnostic code; deterministic output; windows match CONTROL predictions. (4) No regression: baseline store gates per-video + batch metrics; regression test exits 1 on recall/precision drop beyond tolerance. (5) Cross-variant invariants: experiment.compare(CONTROL, variant_disabled_cue) shows F1 delta ? 0, proving the no-regression guarantee for merged-but-disabled new cues.",

```
    "code_ref": "run_all_tests.py, .github/workflows/ci.yml,  
backend/eval/{golden_fixtures,experiment,regression,storage}*.py,  
tests/test_{golden_fixtures,experiment,regression,storage,stub_runner}.py",  
    "kind": "core-contract",  
    "reuse_status": "exists"
```

```
},
```

```
{
```

```
    "topic": "Import Smoke Test (GPU-Stack-Free Baseline)",
```

```
    "detail": "tests/test_import_smoke.py proves the codebase has no SyntaxError/module-  
level ImportError on machines without cv2/torch/numpy (except import-side transitive pulls,  
which are tolerated if never EXECUTED on the replay path). CI lane deliberate excludes GPU  
stack, runs only import smoke to catch breakage early. Repo ethos: offline, mocked,  
deterministic.",
```

```
    "code_ref": "tests/test_import_smoke.py, .github/workflows/ci.yml:54-65",
```

```
    "kind": "core-contract",
```

```
    "reuse_status": "exists"
```

```
}
```

```
],
```

```
"config_or_env_knobs": [
```

```
    "RALLY_DETECTOR_IMPL=stub (env overrides config detector_impl='stub', injected via  
TrajectoryHybridSegmenter._resolve_runner hook)",
```

```
    "detector_impl='stub' in indexing config (config-level seam for detector selection)",
```

```
    "backend=[local|s3|gcs|gdrive] URI scheme + get_storage(config={...}) for storage backend  
selection",
```

```
    "AWS_ENDPOINT_URL_S3 / AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY for S3 (boto3 default  
chain)",
```

```
    "GDRIVE_MOUNT_ROOT env or config mount_root for Google Drive mount point",
```

```
    "pytest --cov=backend --cov-report=term --cov-fail-under=70 for coverage floor (CI floor  
70%, calibrated 2026-06-11)",
```

```
    "mypy.ini lenient baseline with explicit per-module quarantine list for type checking",
```

```
    "ruff.toml for linting (pyflakes/pycodestyle/bugbear)",
```

```
    "Variant.min_crossings, Variant.min_players, Variant.threshold, Variant.merge_gap,
```

```
Variant.tune_threshold, Variant.calibrate for experiment re-parametrization",
```

```
    "golden_fixtures --freeze-synthetic / --check / --refresh-snapshot / --freeze-real CLI for  
fixture lifecycle"
```

```
],
```

```
"notes": "The testing surface reveals a mature, multi-layered regression strategy already in  
place. The core building blocks for proving zero regression across profiles are:\n\nTIER-0  
FOUNDATION (M0): Golden Fixtures (synthetic, CONTROL-only, parse-compare gate on cross-OS).\n- Behavior-locking, not correctness ($6 caveat).\n- Pinned windowing (vt, mg, mwd hardcoded per  
fixture, never imported from SERVED constant).\n- Privacy Tier-0 enforced: FORBIDDEN_KEYS scan  
in expected.json + provenance.\n- Characterization proves post-trajectory determinism; mutation  
test (truncate trajectory, replay, get different segments) proves re-derivation not echo-  
back.\n\nTIER-1 REAL FIXTURE GATE (M2/P1): De-Attribution + Refusal.\n- RefusalError blocks  
unsafe (minors_free, owned, license, label-slug checks).\n- Surrogate fx_hex slugs +  
kind='cleared_real' tags.\n- Time-origin reset metric-invariant (score unchanged, segments shift  
toward 0).\n- Positive privacy assertion: scan for FORBIDDEN_KEYS in expected.json and source  
leaks in provenance.\n- Write-to-temp-then-promote pattern (no partial dirs on refusal).\n\nCORE  
VALIDATION SEAMS (M3.5):\n- Stub detector: deterministic CPU trajectory, no GPU/WSL, feeds  
shared parse/window code.\n- Storage fakes: S3/GCS in-memory clients, GDrive mount-local, no  
real SDKs.\n- Experiment Variant profiles: disabled-cue byte-identical guarantee, CONTROL as  
reference, LOVO-honest train-test split.\n- Regression baseline: per-video  
precision/recall/f1/IoU gates, batch aggregation (micro/macro), exit code contract.\n\nMULTI-  
PROFILE PROOF STRATEGY:\n1. Golden-crossos matrix (3 OSes) runs golden fixtures +  
golden_real_fixtures tests, proving determinism across platforms on CONTROL.\n2. Stub detector +  
storage fakes prove GPU-free e2e (shared code path) with injected fake clients, no production
```

```

SDK calls.\n3. Experiment compare(CONTROL, variant_with_disabled_cue) proves no-regression
invariant (F1 delta ? 0) when merging new features.\n4. Regression baseline gates
precision/recall/f1/IoU per video and aggregates (micro/macro) across batch, exit 0/1/3.\n5.
Import smoke + lint + typecheck gates catch syntax/module-level errors early (no GPU stack
required).\n\nConfiguration knobs are minimalist: env vars (RALLY_DETECTOR_IMPL,
GDRIVE_MOUNT_ROOT, AWS_*) + config dicts (detector_impl, backend, mount_root, Variant fields) +
CLI args (--freeze-synthetic, --check, --refresh-snapshot, --freeze-real, --cov, -k,
--reason).\n\nThe suite is intentionally OFFLINE + MOCKED + DETERMINISTIC. No real GPU, no
network, no WSL, no Drive API. All backends injectable (tests pass fake S3/GCS clients; GDrive
reads tmp_path). All variants reuse the same eval stack (metrics, training, classifier, merge,
regression). The design separates DETECTION (persisted candidate signals) from DECISION (variant
re-scoring), enabling pure-offline A/B, side-by-side, and promotion gates with zero production
risk.\n\nHonest-limits caveat ($6) is baked in: synthetic-tier numbers measure plumbing
correctness, not field quality. A green suite proves the deterministic post-trajectory
transforms are unchanged, not that rally detection is good. Real fixtures (M2) sit behind
counsel sign-off (P2) before rollout."
    }
]

```

Design it. Land the central idea: the core detect+stitch is a PURE function of (input bytes + config) ? outputs; a **ComputeBackend** / deployment profile decides WHERE it runs (local process vs a Cloud Run GPU job) and a **StorageBackend** decides where I/O comes from (local FS / USB / gdrive mount / bucket) ? selected by ONE startup/setup config, so the core is untouched. Cover: the core-unaaffected principle + the exact contract; the profiles (at least **truly-local** and **cloud-serving**, plus any sensible hybrid); the profile-selection/startup-config mechanism; compute placement (detect on GPU + stitch via ffmpeg, and WHY stitch runs on Cloud Run in the cloud profile = metered cost); the end-to-end flows (cloud upload<2GB?segment+stitch on Cloud Run?reel; gdrive/bucket load; truly-local); a reuse-vs-net-new map; a phased execution plan (smallest-safe-first, each = one PR, default-OFF where it touches the core); a RIGOROUS multi-layer testing strategy (unit/contract/integration/e2e; the stub-detector \$0 path; mocked-storage fakes; profile-PARITY tests proving identical core output local-vs-cloud; real-GPU vs mocked; CI gates; and the sample-runs/runlogs-for-posterity convention like DECOUPLED's deploy report); the ADR text (decision/rationale/consequences); open questions for the owner; and risks. Keep edge-auth, per-video cost ledger, DPA/consent, WASB-C3, and NVDEC in scope where relevant.

2. assistant (2026-06-21T02:36:43.548Z)

I'll design this thoroughly. Let me first verify the code-grounded map against the actual codebase, then check the archived DECOUPLED_COMPUTE deploy report convention and key seams.

3. user (2026-06-21T02:36:48.014Z)

```

From https://github.com/Khelsutra/badminton-highlight-indexer
4134721..4957b72 master -> origin/master
Your configuration specifies to merge with the ref 'refs/heads/docs/archive-decoupled-v2'
from the remote, but no such ref was fetched.
---BRANCH---
docs/archive-decoupled-v2

```

4. user (2026-06-21T02:36:48.025Z)

```

60         raise NotImplementedError(
61             f"detector_impl='native' is not supported for the "
62             f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
has a "
63             f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'."
64
65         def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
66             """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.
67
68             ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or

```

```

``indexing.detector_impl``:
69     ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
70     is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
71     in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
72     ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
73     (the WSL runner) so the default production path is unchanged
74     (``docs/DECOUPLED_COMPUTE/``).
75     """
76     impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
77     if impl == "stub":
78         from backend.pipeline.detectors.stub_runner import StubDetectorRunner,
StubConfig
79         logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
80                     self.display_label or "trajectory")
81         return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
82     if impl in ("native", "native_wasb", "wasb_native"):
83         logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
84                     self.display_label or "trajectory")
85         return self._build_native_runner(idx_cfg)
86     return self._build_runner(idx_cfg)
87
88     @staticmethod
89     def _probe_fps_width(video_path: str) -> tuple:
90         """Return (fps, frame_width) for a video, with sensible fallbacks."""
91         cap = cv2.VideoCapture(video_path)
92         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
93         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
94         cap.release()
95         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
96
97     @staticmethod
98     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
99         """Provider-reported exchange count, else a duration-based estimate.
100
101         One canonical implementation ? the twins had drifted (wasb relied on
102         ``int(None)`` raising into the fallback; same result, by accident).
103         """
104         shot_count = segment.get("shuttle_exchange_count")
105         if shot_count is None:
106             return max(3, int(duration / 0.8))
107         try:
108             return int(shot_count)
109         except (ValueError, TypeError):
110             return max(3, int(duration / 0.8))
111
112     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-----
113     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
114                                frame_width: float, video_path: str,
115                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
116         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
117         returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
118         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
119         features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
120         return {
121             "peak_velocity": cw["peak_velocity"],
122             "mean_velocity": cw["mean_velocity"],

```

```

123         "active_frame_count": cw["active_frame_count"],
124         "track_id_count": cw["track_id_count"],
125     }
126
127     def _select_for_handoff(self, windows: List[Dict[str, Any]],
128                             window_stats: List[Dict[str, Any]],
129                             idx_cfg: Dict[str, Any]) -> List[int]:
130         """Indices of the candidate windows that proceed to the AI handoff (and thus may
131         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
132         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
133         Persisted candidates are NOT affected ? they remain the full superset for
134         offline
135         re-scoring."""
136         return list(range(len(windows)))
137
138     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
139                      player_pool: Optional[List[str]] = None,
140                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
141         # override-ignore: the ABC declares the Result contract (Success|Failure)
142         # but every live segmenter still returns a bare list ? the documented
143         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
144         label = self.display_label
145         # --- Sport profile + AI provider wiring (identical across segmenters) ---
146         sport_name = self.config.get("sport", "badminton")
147         try:
148             sport_profile = sport_registry.get(sport_name)
149         except KeyError as e:
150             logger.error(str(e))
151             return []
152
153         idx_cfg = self.config.get("indexing", {})
154         # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
155         the
156         # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
157         is
158         # needed and nothing is uploaded. Used to run the local detector standalone
159         (e.g. to
160         # measure it against the Gemini path, or to avoid the cloud entirely). With
161         # FusionSegmenter the windows are still Gate-A/B-filtered via
162         `_select_for_handoff`.
163         skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
164         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
165         "gemini"))

```

5. assistant (2026-06-21T02:36:58.238Z)

I'm on branch `docs/archive-decoupled-v2`. Let me verify the key seams the design depends on, and check the DECOUPLED deploy-report convention for the runlog format.

6. user (2026-06-21T02:37:00.105Z)

```

225
226     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
227     ok, msg = runner.healthcheck()
228     if not ok:
229         logger.error("%s detector environment not ready: %s", label, msg)
230         return []
231     logger.info("%s detector env OK: %s", label, msg)
232
233     # --- Phase 2: trajectory -> candidate rally windows ---
234     # Trajectory detectors process the whole video in one pass (they carry
235     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
236     t_start = time.time()
237     out_dir = os.path.join("output", self.family)
238     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)

```

```

239         if not csv_path:
240             logger.error("%s produced no trajectory; aborting.", label)
241             return []
242
243         points = runner.parse_trajectory_csv(csv_path)
244         logger.info("Parsed %d trajectory points (%d visible).",
245                     len(points), sum(1 for p in points if p.visible))
246
247         all_candidate_windows = runner.trajectory_to_action_windows(
248             points, fps=fps, frame_width=frame_width, chunk_start=0.0,
249             velocity_thresh=velocity_thresh, merge_gap=merge_gap,
250             min_window_duration=min_window_duration, chunk_index=0,
251             max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
252             window_hard_cap=window_hard_cap,
253             window_inactivity_end=window_inactivity_end,
254             inactivity_rally_thresh=inactivity_rally_thresh,
255             inactivity_min_density=inactivity_min_density,
256             window_blob_fallback=window_blob_fallback,
257             window_blob_factor=window_blob_factor,
258         )
259         logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
260                     label, time.time() - t_start, len(all_candidate_windows))
261
262         if log_candidates:
263             for cw in all_candidate_windows:
264                 logger.info(f"[{label} rally]
265                             {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
266                             f"({cw['end'] - cw['start']:.1f}s) |
267                             frames={cw['active_frame_count']} "
268                             f"peak_v={cw['peak_velocity']:.3f}
269                             mean_v={cw['mean_velocity']:.3f}")
270
271         # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
272         fusion
273         # segmenter adds net_crossings + person-cue features. Computed once, reused for
274         both
275         # persistence and the handoff gate below.
276         window_stats = [
277             self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
278             idx_cfg)
279             for cw in all_candidate_windows
280         ]
281
282         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
283         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
284         if store_candidates:
285             for i, cw in enumerate(all_candidate_windows):
286                 candidate_ids[i] = self.db.add_candidate_segment(
287                     video_id, detector=self.name,
288                     start_time=cw["start"], end_time=cw["end"],
289                     chunk_index=cw["chunk_index"], confidence=min(1.0,
290                     cw["peak_velocity"]),
291                     stats=window_stats[i], detection_params=detection_params,
292                 )
293
294         if not all_candidate_windows:
295             return []
296
297         # --- Phase 3: AI provider handoff over padded candidate clips ---
298         # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
299         Persisted
300         # candidates above stay the full superset ? only the served play_segments are
301         gated.
302         handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
303         idx_cfg)

```

```

294
295     ai_segments: List[dict] = []
296     if skip_ai:
297         # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
298         # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
299         # falls back to the duration-based estimate (no AI exchange count).
300         ai_segments = [
301             {
302                 "start_time": all_candidate_windows[wi]["start"],
303                 "end_time": all_candidate_windows[wi]["end"],
304                 "shuttle_exchange_count": None,
305                 "shot_count_confidence": "LocalNoAI",
306                 "_candidate_id": candidate_ids[wi],
307             }
308             for wi in handoff_indices
309         ]
310         logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
311                     "(no AI handoff).", len(ai_segments))
312     else:
313         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
314         os.makedirs(handoff_dir, exist_ok=True)
315         logger.info("Phase 3: %s validation on %d candidate windows...",
316                     provider_name, len(handoff_indices))
317
318         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
319             w_start = max(0, window["start"] - handoff_padding)
320             w_end = min(video_duration, window["end"] + handoff_padding)
321             w_dur = w_end - w_start
322             w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
323             if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
324                 return []
325             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
326             segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
327                                                  label=f"Handoff {wi+1}")
328             valid = []
329             for seg in segments:
330                 try:
331                     seg["start_time"] = float(seg["start_time"]) + w_start
332                     seg["end_time"] = float(seg["end_time"]) + w_start
333                     seg["_candidate_id"] = candidate_ids[wi]
334                     valid.append(seg)
335                 except (ValueError, TypeError, KeyError) as e:
336                     logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
337             try:
338                 os.remove(w_path)
339             except OSError:
340                 pass
341             return valid
342
343         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
344             futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
345                         for wi in handoff_indices]
346             for future in futures:
347                 ai_segments.extend(future.result())
348         try:
349             os.rmdir(handoff_dir)
350         except OSError:

```

```

351         pass
352         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
353
354         # --- Phase 4: DB population + candidate linking ---

```

7. user (2026-06-21T02:37:00.728Z)

```

278     class SecurityConfig(BaseModel):
279         # When set, /api/process video_path must resolve under this directory (hosted/tenant
280         # mode). Default None preserves the single-user local 'any local file' workflow.
281         allowed_video_root: Optional[str] = None
282
283         # --- Chunked upload (B2, PR-2) ---
284         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
285         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
286         # contain upload_dir or /api/process will reject the uploaded paths.
287         upload_dir: str = "output/uploads"
288         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
289         # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
290         max_upload_mb: int = 4096
291         max_part_mb: int = 95
292         allowed_upload_extensions: List[str] = ["mp4", "mov"]
293
294
295     class StorageConfig(BaseModel):
296         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
297         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
298         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
299         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
300         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
301         (boto3/google auth chains supply credentials). See ``backend/storage/``. """
302         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
303         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
304         output_root: str = ""
305         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
306         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
307         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
308         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
309         # the legacy flat layout is used unchanged.
310         single_folder_per_video: bool = False
311         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
312         workspace_root: str = ""
313         # Cloud namespace prefix under the root so per-video folders sit tidily beside
314         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
315         workspace_prefix: str = "workspaces"
316         local: dict[str, Any] = Field(default_factory=dict)
317         s3: dict[str, Any] = Field(default_factory=dict)
318         gcs: dict[str, Any] = Field(default_factory=dict)
319         gdrive: dict[str, Any] = Field(default_factory=dict)
320
321
322     class AppConfig(BaseModel):
323         """Root configuration object.
324
325         All runtime configuration should be accessed via an instance of this class

```

```

326         (or the loader) rather than raw dict lookups.
327         """
328
329         sport: Sport = "badminton"
330         ai_provider: str = "gemini"
331         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
332         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
333         ai_model: str = "gemini-flash-latest"
334
335         validation: ValidationConfig = Field(default_factory=ValidationConfig)
336         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
337         output: OutputConfig = Field(default_factory=OutputConfig)
338         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
339         security: SecurityConfig = Field(default_factory=SecurityConfig)
340         logging: LoggingConfig = Field(default_factory=LoggingConfig)
341         storage: StorageConfig = Field(default_factory=StorageConfig)
342
343         # Allow extra keys during transition so existing config.json files do not break.
344         model_config = {
345             "extra": "allow",
346             "validate_assignment": True,
347         }
348
349         # Temporary migration shim for legacy dict-style access (config.get("section",
{ }).get(...)).
350         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
351         # attribute access / the typed config. A nested *section* is returned as a plain
dict
352         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
353         # section model instead breaks it (a model has no .get), which bit every validator
on
354         # real runs. Scalar leaves are returned as-is.
355         def get(self, key: str, default: Any = None) -> Any:
356             _MISSING = object()
357             val: Any = _MISSING
358             if key in type(self).model_fields or hasattr(self, key):
359                 val = getattr(self, key, _MISSING)
360             if val is _MISSING:
361                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
362                 # discovered dynamically from the model fields, so a future top-level
section
363                 # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
364                 for fname in type(self).model_fields:
365                     section = getattr(self, fname, None)
366                     if isinstance(section, BaseModel) and hasattr(section, key):
367                         val = getattr(section, key)
368                         break
369             if val is _MISSING:
370                 return default
371             if isinstance(val, BaseModel):
372                 return val.model_dump(mode="json")
373             return val
374
375         def __getitem__(self, key: str) -> Any:
376             val = self.get(key)
377             if val is None and key not in type(self).model_fields:
378                 # Re-raise like a real dict for unknown top-level
379                 raise KeyError(key)
380             return val
381
382

```

```

383     # Convenience type alias for callers that want to annotate
384     Config = AppConfig
385

```

8. assistant (2026-06-21T02:37:04.319Z)

The map is accurate. Let me verify the worker endpoint, storage backend ABC, and the DECOUPLED deploy report convention (for the runlog format).

9. user (2026-06-21T02:37:05.747Z)

```

80         "video_id": video_id,
81         "status": status,
82         "segment_count": len(segments),
83         "segments": [{"start": float(s.get("start_time", 0.0)),
84                       "end": float(s.get("end_time", 0.0))} for s in segments],
85     }, f, indent=2)
86
87
88     def cmd_infer(args: argparse.Namespace) -> int:
89         config = load_config(args.config) if args.config else load_config()
90         _apply_overrides(config, args.set)
91         storage_cfg = config.storage.model_dump()
92
93         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
94         os.makedirs(scratch, exist_ok=True)
95         config.output.output_dir = scratch
96
97         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
98         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
99         config=storage_cfg)
100         print(f"[worker] pulled {args.video_uri} -> {local_video}")
101
102         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
103         byte-identical).
104         video_id = compute_video_id(local_video)
105
106         # 3. VALIDATE (same registry as the main CLI).
107         val_results, val_context = validator_registry.run_all_with_context(local_video,
108         config)
109         failures = [r for r in val_results if not r.passed]
110         db = Database(os.path.join(scratch, config.output.db_name))
111         db.add_video(video_id, local_video, "failed" if failures else "processed",
112         [r.model_dump() for r in val_results])
113
114         segments: List[dict] = []
115         segmenter_actual = None
116         segmentation_ran = False
117         status = "failed"
118         try:
119             if failures:
120                 print("[worker] validation FAILED: "
121                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
122                 return 2
123             seg_name = args.segmenter or config.indexing.default_segmenter
124             segmenter_cls = segmenter_registry.get(seg_name)
125             if not segmenter_cls:
126                 print(f"[worker] unknown segmenter: {seg_name!r} "
127                 f"(have {list(segmenter_registry.list_available())})")
128                 return 2
129             segmenter_actual = seg_name
130             result = segmenter_cls(db, config).process_video(video_id, local_video,
131             max_frames=args.max_frames)
132             segmentation_ran = True

```

```

129         if isinstance(result, Failure):
130             print(f"[worker] segmenter FAILED: {result.message}")
131             return 3
132         segments = result.value if hasattr(result, "value") else result
133         status = "success"
134     finally:
135         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
136         emit_indexer_report(
137             db=db, video_id=video_id, video_path=local_video, config=config,
138             val_results=val_results, val_context=val_context,
139             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
140             was_reingest=False, segmentation_ran=segmentation_ran,
141         )
142
143     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
144     out_local = os.path.join(scratch, "outputs", video_id)
145     os.makedirs(out_local, exist_ok=True)
146     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
147     _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
148
149     out_prefix = args.out_uri.rstrip("/")
150     pushed = []
151     for fn in sorted(os.listdir(out_local)):
152         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
153         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
154         pushed.append(uri)
155
156     print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
157     for u in pushed:
158         print("    ", u)
159     return 0
160
161
162     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
163     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
164     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
165
166
167     def _probe_video_fps_width(path: str):
168         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
169

```

10. user (2026-06-21T02:37:06.945Z)

```

docs\archives\past_projects\DECOUPLED_COMPUTE\01-EXISTING-PLANS-AND-GAP.md
docs\archives\past_projects\DECOUPLED_COMPUTE\02-COMPUTE-MAP-AND-CONTRACT.md
docs\archives\past_projects\DECOUPLED_COMPUTE\03-PLATFORM-OPTIONS.md
docs\archives\past_projects\DECOUPLED_COMPUTE\04-BUILD-AND-TEST-PLAN.md
docs\archives\past_projects\DECOUPLED_COMPUTE\05-COST-ATTRIBUTION.md
docs\archives\past_projects\DECOUPLED_COMPUTE\06-RISKS-AND-OPEN-QUESTIONS.md
docs\archives\past_projects\DECOUPLED_COMPUTE\07-COMPUTE-ABSTRACTION.md
docs\archives\past_projects\DECOUPLED_COMPUTE\DEPLOY_REPORT_GCP_2026-06-20.md
docs\archives\past_projects\DECOUPLED_COMPUTE\HANDOFF.md
docs\archives\past_projects\DECOUPLED_COMPUTE\README.md

```

11. user (2026-06-21T02:37:06.969Z)

```

backend\storage\gcs.py
backend\storage\local.py
backend\storage\s3.py

```

```
backend\storage\__init__.py
backend\storage\base.py
backend\storage\ref.py
backend\storage\gdrive.py
```

12. user (2026-06-21T02:37:11.327Z)

```
1      # Deployment report ? first real GPU run on GCP (L4, asia-south1), 2026-06-20
2
3      **What this is.** The first end-to-end **non-mock** run of the decoupled pipeline on a
4      rented cloud GPU:
5      the M3.5 native WASB runner extracting REAL shuttle trajectories on a GCP **NVIDIA L4**,
6      driving the
7      bucket-driven `worker infer` / `worker train`. Closes M1/M2/M3.5 of `04-BUILD-AND-TEST-
8      PLAN.md` on cloud
9      hardware (per ADR-011 scope). Written for posterity + to make the next deploy one-shot.
10
11     ## TL;DR
12     - ? **Decoupling works + the GPU is genuinely used.** `worker infer
13     detector_impl=native` ran the full
14     pipeline on the L4 (gcs:// in ? real WASB extraction on GPU ? rallies ? gcs:// out):
15     **22 rallies** on
16     the 180 s smoke clip.
17     - ? **M3.5 native runner runs on a real cloud GPU** (NVIDIA L4 23 GB) ? same
18     `wasb_infer.py`, device cuda.
19     - ? **Cross-GPU parity is NOT byte-identical** (L4 vs the 2070): 684/1195 frames differ
20     by **sub-pixel**
21     amounts (cuDNN across architectures) + a few visibility flips ? the M3.5 gate belongs
22     at the
23     **rally/window level**, not CSV bytes. (Same-GPU IS byte-exact: the 2070 gave
24     1194/1195.)
25     - ? **M2 real-trajectory `worker train` FINALIZED** (Run 2, later 2026-06-20, an L4 in
26     **asia-southeast1
27     / Singapore** ? Mumbai was fully GPU-stocked-out): real-detector **leave-one-video-
28     out** over
29     GX030094_proxy + mahadevpura-1 ? Gen-0 bundle pushed to **`weights/0.1.0-l4/`**
30     (`weights.json` +
31     `manifest.json`). `mean_f1 0.045` is an **honest n=2 plumbing result** (proves the
32     real-detector train
33     path e2e on cloud GPU), NOT a quality model ? full-corpus training is the next quality
34     step;
35     `gate.ship=False` (no floor / uncalibrated / WASB-C3, all expected).
36     - ? **Highlight reel** of the M1 rallies pushed ?
37     `outputs/05e0e577?/highlight_smoke_l4.mp4` (22 rallies,
38     94 s). **Smoke-tested:** valid h264 1080p, 95.2 s, **clean full decode (`ffmpeg -f
39     null`, exit 0)** , and a
40     5-frame contact sheet **visually confirms real badminton rallies** (court, net,
41     players mid-rally).
42     - ? **DigitalOcean DROPPED for compute (ADR-013, 2026-06-20).** DO has **0 GPU Droplet
43     sizes** (not
44     enabled) + charges newer customers' cards for GPU (not credit-funded); Paperspace is
45     subscription-gated.
46     **GCP is the sole compute stack** (training + serving) going forward.
47
48     ## Resources provisioned (project `gen-lang-client-0306026826`, billing "Khelsutra
49     Billing")
50     | Resource | Identifier | Console |
51     |---|---|---|
52     | GCS bucket | `gs://khelsutra-rally-corpus` (asia-south1) | storage/browser |
53     | Service account | `rally-worker@?` ? storage.objectAdmin ** + monitoring.metricWriter +
54     logging.logWriter** (added for Ops Agent) | iam-admin/serviceaccounts |
55     | Budget | `rally-100usd-guard` ($100, 50/90/100%) | billing budgets |
56     | GPU quota | `GPUS_ALL_REGIONS` ? **1** (granted after a Cloud Quotas API request) |
57     iam-admin/quotas |
58     | GPU VM | `rally-gpu` ? **L4 / g2-standard-4 / asia-south1-c**, image `common-
```

```

cu129-ubuntu-2204-nvidia-580` | compute/instances |
37 | Ops Agent | installed on the VM ? GPU/NVML metrics + logs in Observability | VM ?
Observability |
38
39 Bucket layout staged: `models/wasb_badminton_best.pth.tar`, `labels/*.rallies.csv` (7),
`videos/`
40 (smoke + the 2 smallest corpus videos), `outputs/`, `weights/`.
41
42 ## Run results
43 - **M3.5 parity** (`wasb_infer.py --device cuda`, smoke clip, limit 1200): produced
44 `outputs/parity/native_smoke_l4.csv` (1200 frames). vs the 2070 WSL baseline:
**511/1195 identical,
45 684 differ sub-pixel** (e.g. 968.01?967.87 px), visible 682 vs 684. ? parity within
~tenths-of-a-px
46 cross-GPU; **not** byte-exact across GPUs.
47 - **M1 decoupled infer** (`worker infer detector_impl=native`, smoke clip, full 5393
windows on GPU):
48 **22 rallies** ? `outputs/<md5>/intervals.csv` + `report.json` pushed.
49 - **M2 real-trajectory train** ? ? FINALIZED (Run 2, asia-southeast1 / Singapore L4).**
`worker train
50 --trajectory-source detector --version 0.1.0-l4` with
`indexing.wasb.stream_video=false` +
51 `timeout_sec=10800`: pulled the 2 label?video pairs (GX030094_proxy + mahadevpura-1)
from the bucket,
52 ran **real native-WASB extraction on the L4** per video (GX030094_proxy = 21 197
frames; trajectory
53 CSVs written, then the gen0 harness trained LOVO) and **pushed the Gen-0 bundle** ?
54 `gs://khehsutra-rally-corpus/weights/0.1.0-l4/{weights.json,manifest.json}` (`rc=0`).
55 **Held-out (LOVO, n=2):** GX030094_proxy per-frame AUC 0.782 / F1 0.091; mahadevpura-1
AUC 0.582 /
56 F1 0.000 ? **mean F1 0.045**. This is an **honest small-n plumbing result** (the real-
detector train
57 path runs e2e on cloud GPU and emits a versioned, provenance-stamped bundle), **not**
a quality model ?
58 real quality needs the full corpus (Next steps). `manifest.json`:
`label_source=human`,
59 `gate.ship=False` (no floor / ship-gate uncalibrated #172 / WASB-C3). ?
`git_sha=unknown` in the
60 manifest ? the box ran from a `git archive` tarball (no `.git`); pass the SHA via
env/VERSION next time.
61
62 ## Findings / gotchas (all fixed or noted)
63 1. **conda Terms-of-Service gate** ? modern conda blocks non-interactive `conda create`
64 (`CondaToSNonInteractiveError`). **Fixed in `gpu_setup.sh`** (`conda tos accept ?`).
65 2. **WASB-SBDT `requirements.txt` is incomplete** ? `src/` imports `hydra`, `pandas`,
etc. it omits.
66 **Fixed in `gpu_setup.sh`** (explicit `hydra-core omegaconf pandas opencv-python-
headless pillow scipy
67 tqdm pyyaml matplotlib numpy<1.24`).
68 3. **T4 capacity stockout** in `asia-south1-a` ? **L4 in `-c`** landed (g2-standard-4;
L4 is built into g2,
69 no `--accelerator` flag). Try L4/other zones when T4 is unavailable.
70 4. **Spot does NOT bypass `GPUS_ALL_REGIONS`** (tested) ? both on-demand + Spot need the
quota.

```

13. user (2026-06-21T02:37:11.720Z)

```

1 """The ``StorageBackend`` ABC ? one interface for local + object-store + Drive.
2
3 Mirrors the codebase's ABC+Registry idiom (cf. ``SegmenterRegistry``). The bucket
backends
4 live in sibling modules and lazy-import their cloud SDK so ``import backend.storage``
never
5 pulls ``boto3`` / ``google-cloud-storage`` (``gdrive`` is pure local-FS over a mounted
Drive ?

```

```

6      no SDK). See ``PLATFORM_ARCHITECTURE.md`` §3.1 and ``docs/DECOUPLED_COMPUTE/``.
7      """
8      from __future__ import annotations
9
10     import os
11     from abc import ABC, abstractmethod
12     from typing import Iterator
13
14     from backend.storage.ref import StorageRef, StorageStat
15
16
17     class StorageBackend(ABC):
18         """Backend-agnostic object storage. ``fetch_to_local`` + ``put`` are the two the
19         worker dispatcher needs; the rest support listing/checkpointing/serving."""
20
21         @abstractmethod
22         def fetch_to_local(self, ref: StorageRef, dst: "str | os.PathLike[str] | None" =
None,
23
24             *, overwrite: bool = False) -> str:
25             """Make the object available as a local file; return its local path. For the
26             ``local`` backend this is an identity passthrough (the object is already
27             local)."""
28
29         @abstractmethod
30         def put(self, local_path: "str | os.PathLike[str]", ref: StorageRef, *,
31             content_type: "str | None" = None) -> StorageRef:
32             """Upload/copy a local file to ``ref``; return ``ref``."""
33
34         @abstractmethod
35         def exists(self, ref: StorageRef) -> bool: ...
36
37         @abstractmethod
38         def stat(self, ref: StorageRef) -> StorageStat: ...
39
40         @abstractmethod
41         def list(self, prefix: StorageRef) -> Iterator[StorageRef]:
42             """Yield a ``StorageRef`` per object under ``prefix``."""
43
44         @abstractmethod
45         def delete(self, ref: StorageRef) -> None: ...
46
47         def signed_url(self, ref: StorageRef, *, ttl: int = 3600, method: str = "GET") ->
str:
48             """A time-limited URL handing the bytes to a browser/LLM. Not every backend can
49             sign (``local`` cannot; Drive only emulates) ? callers must treat
50             ``NotImplementedError`` as expected and fall back to ``fetch_to_local``."""
51             raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")

```

14. user (2026-06-21T02:37:12.689Z)

```

1      # 07 · Compute abstraction ? segmenting GPU vs CPU + a spec-aware device layer
2
3      **Status:** PROPOSAL (design) · **Date:** 2026-06-20 · code-verified 2026-06-20
4
5      > "Do we have a GPU Runner interface? How do we segment GPU vs CPU work ? cleanly
abstracting the GPU while
6      > leaving room for customizations based on the GPU spec?" Answer: **not yet as a single
interface**, but the
7      > seam and a spec-probe already exist. This doc defines the missing **device layer** and
the **GPU/CPU
8      > segmentation rule** ? small, and consistent with the codebase's ABC+Registry idiom.
9
10     ## 1. What exists today (honest inventory)
11

```

```

12      | Concern | Today | Verdict |
13      |---|---|---|
14      | Global GPU switch | `use_gpu: bool = True`
([`config/models.py:119`](../../backend/config/models.py)) | Coarse ? one flag for the whole
run. |
15      | torchvision person detector | `self.device = "cuda" if use_gpu and
torch.cuda.is_available() else "cpu" + warn/fallback
([`person_detector.py:74-76`](../../backend/pipeline/detectors/person_detector.py)) | **Clean
fallback, but no spec-awareness** (no VRAM/batch/precision logic; per-frame `.to(device)`). |
16      | WASB detector | hardcoded `device='cuda'`, **no CPU fallback** | Inconsistent ? a
portability/robustness gap. |
17      | TrackNet (TF) | GPU only via WSL; healthcheck requires a visible GPU | WSL-bound (see
[02](02-COMPUTE-MAP-AND-CONTRACT.md), [04 M3.5](04-BUILD-AND-TEST-PLAN.md)). |
18      | `DetectorRunner` ABC | abstracts the **task** (run_predict/healthcheck)
([`detectors/base.py`](../../backend/pipeline/detectors/base.py)) | Device handling lives
**inside** each concrete runner, not shared. |
19      | `ComputeBackend` (planned) | abstracts **where** a job runs (local/hosted/byo_worker)
| Orthogonal to *how* GPU work executes ? don't conflate. |
20      | GPU spec probe | `gpu_name()` + `gpu_stats()` (name, VRAM used/free via `nvidia-smi`)
([`run_telemetry.py`](../../backend/utils/run_telemetry.py)) | **The seed of a `DeviceSpec` is
already here.** |
21      | Macro GPU/CPU split | detect-once-on-GPU ? persist CSV/features ? re-score on CPU
(EXPERIMENT_HARNESS, GOLDEN_DATA) | **Already the right segmentation ? at the artifact
boundary.** |
22
23      **Conclusion:** there is no unified "GPU Runner." Device selection is scattered across
components with three
24      different styles (global flag, clean-fallback, hardcoded-cuda). The fix is one small
shared layer.
25
26      ## 2. Two orthogonal axes ? keep them separate
27
28      ...
29      WHERE the job runs          ?   HOW a GPU stage executes on this box
30      (ComputeBackend ? coarse,    ?   (DeviceContext ? fine, spec-aware,
31      per-job: local | DO droplet ?   per-stage: probe the actual GPU ?
32      | serverless | byo_worker) ?   resolve batch/precision/chunk knobs)
33      picks the machine ???????? on that machine, DeviceContext.detect()
34      ?   probes the real GPU and tunes the work
35      ...
36
37      `ComputeBackend` answers *which machine*; **`DeviceContext` answers *given this
machine's GPU, how do I run
38      this stage*.** A serverless L4, a DO RTX-4090 Droplet, and CPU CI each get correct knobs
**automatically**,
39      with per-spec overrides available. Conflating the two is the main design trap.
40
41      ## 3. The GPU/CPU segmentation rule
42
43      **Classify every pipeline stage; GPU work is concentrated in the detection substrate.**
44
45      | Stage | Class | Notes |
46      |---|---|---|
47      | Validators (format/res/pts/scene/blur/relevance) | **CPU** | ffmpeg + OpenCV/numpy |
48      | ffmpeg chunk extraction / stitching | **CPU** (`libx264`); GPU optional (`nvenc`) | no
driver-codec dependency by default |
49      | **Person detection** (torchvision Faster R-CNN) | **GPU** (auto CPU-fallback) |
`DeviceContext` consumer |
50      | ByteTrack tracking | **CPU** | runs on the detections |
51      | **Shuttle trajectory** (WASB torch / TrackNet TF) | **GPU** | `DeviceContext`
consumer; the WSL?native target |
52      | Windowing / candidate merge | **CPU** (numpy) | model-agnostic |
53      | AI handoff (Gemini) | **network I/O** | no local GPU; `ai_max_workers` parallelism |
54      | DB writes, eval/scoring, calibration | **CPU** (numpy) | device-agnostic |
55      | Gen-0 head training | **tiny GPU/CPU** | ~4.5 GB, minutes |

```

```

56
57      **The rule:** *a GPU-bound stage is a pure, restartable unit whose only job is `frames ?
artifact` (the
58      `Frame,Visibility,X,Y` CSV or a feature JSON). Everything downstream is device-agnostic
CPU that reads that
59      artifact.* This single principle delivers three things at once:
60      1. **Clean GPU/CPU segmentation** ? only the detection substrate touches a
`DeviceContext`.

```

15. assistant (2026-06-21T02:37:17.090Z)

I have everything I need. The map is accurate, the DECOUPLED deploy-report convention is clear, and the ComputeBackend/DeviceContext separation is already designed. Let me check the project doc template structure and the artifacts/ dir mentioned in git status.

16. user (2026-06-21T02:37:20.174Z)

```

total 1084
drwxr-xr-x 1 avidu 197609      0 Jun 20 23:51 .
drwxr-xr-x 1 avidu 197609      0 Jun 21 06:18 ..
-rw-r--r-- 1 avidu 197609 16460 Jun 20 14:42 appliance_design.json
drwxr-xr-x 1 avidu 197609      0 Jun 20 00:44 landing
-rw-r--r-- 1 avidu 197609 18149 Jun 18 13:25 long_rally_lens_n15.json
-rw-r--r-- 1 avidu 197609 3870 Jun 18 13:25 long_rally_lens_n15.txt
drwxr-xr-x 1 avidu 197609      0 Jun 20 01:12 multishuttle_142
-rw-r--r-- 1 avidu 197609 5602 Jun 19 22:23 pr245_code_review.md
-rw-r--r-- 1 avidu 197609 4028 Jun 19 22:23 pr245_fixes_plan.md
-rw-r--r-- 1 avidu 197609 8520 Jun 19 22:25 pr245_review.json
-rw-r--r-- 1 avidu 197609 5909 Jun 19 22:22 pr245_review.md
drwxr-xr-x 1 avidu 197609      0 Jun 21 06:33 promo
drwxr-xr-x 1 avidu 197609      0 Jun 10 15:54 rally_tal_gen0
drwxr-xr-x 1 avidu 197609      0 Jun 20 23:52 review
drwxr-xr-x 1 avidu 197609      0 Jun 20 07:59 site_deploy
-rw-r--r-- 1 avidu 197609 15148 Jun 20 14:40 so.json
-rw-r--r-- 1 avidu 197609 15485 Jun 20 14:40 so.py
-rw-r--r-- 1 avidu 197609 912462 Jun 16 08:50 tridet.pdf
-rw-r--r-- 1 avidu 197609 46340 Jun 16 08:50 tridet.txt
---TEMPLATE---
C:/Users/avidu/Projects/badminton-highlight-indexer/docs/PROJECT_DOC_TEMPLATE.md
<!--
PROJECT DOC TEMPLATE ? copy this file to docs/<YOUR_PLAN>.md and fill it in.

```

A "tracked project doc" is how this repo carries substantial design/project work: it can be ITERATED on, its progress TRACKED, and it is MARKED DONE against explicit deliverables, then archived. Standardizes the emergent pattern in docs/RALLY_QUALITY_RESEARCH.md §7 and docs/GOLDEN_REGRESSION_FIXTURES.md.

Rules of thumb:

- §7 (the progress tracker) is the SOURCE OF TRUTH ? one row per independently-shippable PR.
 - Keep it HONEST: tag each claim [verified] (checked against code), [researched] (needs sign-off), or [design] (proposed). The doc reflects real shipped state, never aspirational.
 - It POINTS to detail (code anchors, research artifacts), it does not duplicate it.
 - Sections marked OPTIONAL are included only when relevant (e.g. Foundation/Threat-model for research- or security/privacy-heavy work). Delete sections that do not apply.
 - Move to docs/archives/ once Status = DONE (per the archive policy: only terminal-state work).
- >

<Project / Plan Title>

```

> **Status:** `DRAFT` ? `` . **Owner:** `` . **Created:**
`<YYYY-MM-DD>` . **Last updated:** `<<YYYY-MM-DD>`
> **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE)
> **Tracking anchors:** §7 progress tracker is the source of truth; indexed in `docs/README.md`;

```

pointer in memory `current-state` (and `docs/NEXT\_STEPS.md` once work starts).
> **Relation to existing docs:** `<extends | supersedes | peer-of ?>`
> **Honesty note:** mark each claim `[verified]` / `[researched]` / `[design]`. Not legal/financial advice where applicable.

0. TL;DR

<25 sentences: the goal, the approach, and the single most important caveat. Call out any finding that is already actionable on its own.>

1. Problem & goal

<Why now; the concrete user-facing outcome; what "good" looks like. Link the code/docs a reader must read to get current.>

2. Decisions locked

<Table ? capture answered questions so they are not relitigated.>

#	Decision	Source / date	Implication

D1	?	?	?

17. user (2026-06-21T02:37:21.494Z)

143:### 3.2 `ComputeBackend` / `ExecutionBackend` registry
230: status (queued|running|done|failed), progress, compute_backend, result_ref, error,
302: rally/segment/event base VLM via quality + the ComputeBackend;
372: **ComputeBackend** / BYO-compute seam (\$3.2). **Likely** a better fit than Tinker for
410:2. **Fine-tune** via ms-swift on a **ComputeBackend** GPU (\$3.2), run as an orchestration
478:| **P3** | **ComputeBackend** registry + BYO-compute | `local`/`hosted`/`byo\_worker`;
pull-based worker agent; control/data-plane split |

18. assistant (2026-06-21T02:37:24.482Z)

I have full grounding now. Let me read the ComputeBackend registry section of PLATFORM_ARCHITECTURE to align my ADR/design with the existing naming.

19. user (2026-06-21T02:37:25.677Z)

143 ### 3.2 `ComputeBackend` / `ExecutionBackend` registry
144 Where the pipeline actually runs for a given job.
145
146 - **local** ? in-process / local subprocess (today's behavior; dev + self-host).
147 - **hosted** ? our GPU VM pool (the SaaS default).
148 - **byo_worker** ? the tenant runs a **pull-based worker** (a small agent) on their
own
149 VM/GPU. **Critical security property:** the worker **pulls** jobs scoped to its tenant
150 from the control plane, pulls input from the tenant's **own** storage, runs the
151 pipeline, and writes artifacts + results back ? so **the orchestrator never holds the**
152 **user's cloud credentials** and needs **no inbound access** to their network. This
153 mirrors the **self-hosted CI runner** pattern (GitHub Actions runners use a 50s
154 outbound-only HTTPS long-poll). This pull-based runner is the **cheapest, most**
155 **credential-safe v1** for BYO-compute.
156 - For users who want the platform to **provision GPUs** in their **own** cloud account on
157 demand (an academy with an AWS account but no standing GPU box), add **SkyPilot**
158 (BYOC: everything launches in the user's accounts/VPCs; creds stay with them).
159 **Modal** is the managed fallback for users wanting zero infra (accepting that raw
160 video then flows through Modal).
161 - **Operational / support burden** (a real adoption risk). For the primary beachhead
?
162 Indian badminton **academies/coaches** ? asking them to **run and maintain a worker** on
163 their own VM/GPU is a meaningful ongoing load (updates, monitoring, firewall/NAT, the
164 "it stopped after our IT change" ticket). So: **default academy users to `hosted`**,

```

and
165     treat `byo_worker`/SkyPilot as a **power-user / enterprise / privacy-sensitive tier**,
166     not the academy default. If/when BYO ships, plan for **auto-updating, self-healing
167     agents + health heartbeats + clear worker status in the UI**, and budget the support
cost.
168     - **Middle tier to consider ? "zero-infra BYO":** a managed-but-isolated path where the
169     **storage** stays in the user's bucket (signed URLs) while compute runs on **our**
GPUs
170     in a **strictly isolated, per-tenant, ephemeral sandbox** (nothing persists our side).
171     This gives the BYO *privacy* benefit (raw video never persists on our infra)
**without**
172     the user running a worker ? a pragmatic default between full-hosted and full-BYO.
173
174     The pipeline code is unchanged; only *where it's invoked* and *how it gets bytes* vary.
175
176     ### 3.3 Reuse, don't reinvent
177     The existing `segmenter` / `provider` / `validator` / `sport` / `annotation` registries
178     are the proven precedent and stay as the **pipeline** layer. The `annotation` registry +
179     `candidate_segments` table are also the natural seam for the **coach-correction / golden
180     -set** loop in the assistant-coach phase ($6, P4).
181
182     ## 4. Multi-tenant data model & serving layer
183
184     ### 4.1 Entities ? generic core, with the coach model as an optional overlay
185     The foundational schema is deliberately **domain-neutral** so we don't overfit to the
186     coach mindset ? the coach/athlete framing is a **market-fit hypothesis that will
iterate**
187     (see C13 / PMF), not a load-bearing assumption. **Generic core:**
188     ```
189     User ??< VideoCollection ??< Video ??< Highlight ??< HighlightMetadata
190             ???< PlaySegment ??< Event
191     Video / Highlight ??< Job (processing record: status, compute backend, timings)
192     ```
193     - **Generic & re-targetable:** a `VideoCollection` groups a user's videos (a season, an
194     event, a "folder") with no assumption of a coach or academy ? a power user, a coach,
or a
195     parent are all just `User`s with collections.
196     - **Highlights are first-class** (today compile only writes a file): a `highlights`
table
197     (segment ids + output `StorageRef` + params + created_at) powers "see past videos and
198     highlights by history," and **`HighlightMetadata`** carries the **rich semantic tags**
199     (shot type, rally outcome, etc. ? see $6 metadata stage) plus titles / ratings /
notes.
200     - **`video_id` stays the content MD5** (dedup within a tenant), scoped by `tenant_id`.
201     - **Coach overlay (optional, additive):** for the academy/coach GTM, layer
202     `Org/Academy ? Coach ? Athlete` *on top* (e.g. a `Membership`/role table + an
`athlete_id`

```

20. assistant (2026-06-21T02:37:30.905Z)

I now have complete grounding. The design aligns with the existing `ComputeBackend` registry (local | hosted | byo_worker), the `StorageBackend` ABC, the `DeviceContext` separation, and the DECOUPLED deploy-report convention. Let me compose the structured output.

21. user (2026-06-21T02:41:02.514Z)

Structured output provided successfully

22. assistant (2026-06-21T02:41:18.962Z)

Design delivered via StructuredOutput. Summary of what I grounded and produced:

****Verified against code**** (not just the supplied map):

- The pure-core contract ? `DetectorRunner.run\_predict?CSV` (base.py:164-192),

`trajectory\_to\_action\_windows` (tracknet_runner.py:281-298), `VideoCompiler.compile\_highlights` (compiler.py:303-414).

- The single compute seam `\_resolve\_runner` (trajectory_hybrid.py:65-86) and the hardcoded `output/` callers (lines 237, 313).
- `StorageBackend` ABC (backend/storage/base.py) + 4 backends; the already-profile-agnostic worker `cmd\_infer` (worker/__main__.py:88-159).
- The existing `ComputeBackend` registry naming (local|hosted|byo_worker) in PLATFORM_ARCHITECTURE.md §3.2 and the `DeviceContext` separation in the archived DECOUPLED 07-COMPUTE-ABSTRACTION.md.
- The DECOUPLED deploy-report convention (DEPLOY_REPORT_GCP_2026-06-20.md) ? including the load-bearing honest finding that ****cross-GPU detection is NOT byte-identical**** (684/1195 frames differ sub-pixel), which drives the decision to enforce parity at the window level, not CSV bytes.

****Central idea landed:**** core = pure fn of (bytes + config); a profile/ComputeBackend picks WHERE it runs, a StorageBackend picks WHERE I/O comes from, selected by ONE startup config ? core untouched.

****Covered:**** core-unaaffected principle + exact contract; profiles (truly-local, cloud-serving, cpu-mock, deferred byo_worker hybrid); profile-selection/startup-config mechanism (new `deployment`/`compute\_target` + presets + env auto-detect + input_backend); compute placement with the WHY-stitch-on-Cloud-Run metering rationale; the three E2E flows; reuse-vs-net-new map; a 10-milestone smallest-safe-first plan (one PR each, default-OFF on core-touching changes); a multi-layer testing strategy with the profile-parity proof, \$0 stub path, mocked-storage fakes, real-GPU-vs-mocked split, CI gates, and the runlogs-for-posterity convention; ADR-014 text; open questions; and risks. Edge-auth, the per-video cost ledger, DPA/consent, WASB-C3, and NVDEC/NVENC are all in scope where relevant.

Note for the human author: I did not write any files (per instructions). The eventual project dir would be `docs/COMPUTE\_DECOUPLED\_SERVING/` with a `runlogs/` subdir, mirroring the archived DECOUPLED layout.